Parámetros: SPP, variantes, WFC, CSCV y WFM

AlgoProject — manual de uso. Actualizado el 2026-10-12 con `tools/manual.py`.

1. 33. Configurar los SPP en SQX — las dos tareas de permutación
2. 8. SPP — sacar a Python todo lo que deja el Sys. Param Permutation
3. 9. Diccionario — todo lo que se puede extraer, campo por campo
4. 15. SPP Ultra — ¿merece esta estrategia las 5.000 variantes?
5. 18. Variantes — fabricar el lote de variantes de una madre
6. 37. El retest de las variantes para el WFC y el CSCV
7. 19. Walk forward correlation (paso 17) — ¿elegir los parámetros por el pasado sirve fuera?
8. 107. Calibración del paso 17 — ¿cuántas veces dice PASA de la nada, y cuántas NO PASA de algo bueno?
9. 25. CSCV — ¿te fías de tu forma de elegir parámetros?
10. 39. La nube de parámetros — ¿qué parámetros mueven el resultado, y se puede leer ese ranking?
11. 52. Superficies por mercado — ¿lo que va mejor en el mercado principal va mejor en los demás?
12. 34. Configurar la Walk Forward Matrix en SQX
13. 9. Walk-Forward Matrix — sacar a Python todo lo que guarda el cross-check WFM
14. 14. Walk-Forward Matrix — ¿reoptimizar gana en el tramo que nadie miró?
15. 110. La lectura única de oos2 — el último corte de la cola

33. Configurar los SPP en SQX — las dos tareas de permutación

> **Ojo con el nombre.** Esta página es la mitad de SQX del paso 15: **dejar las tareas `SPP IS` y `SPP OOS` configuradas** dentro de un custom project. Sacar el perfil que producen es `09-optimizacion.pdf › 08-spp`, el diccionario de sus columnas es `09-optimizacion.pdf › 09-diccionario-spp`, y leerlo para decidir si una

> estrategia merece 5.000 variantes es `09-optimizacion.pdf › 15-sppultra` (paso 16), que incluye las superficies por pareja de parámetros (dos desplegables, eje X y eje Y, con θ_0 marcado).

Qué pregunta responde

El SPP —*System Parameter Permutation*— mueve **cada parámetro de la estrategia** arriba y abajo y vuelve a correr el backtest en cada combinación. Lo que queda es un mapa: si el beneficio sólo existe en el valor exacto que salió del generador, la estrategia está clavada en un pico y no hay nada debajo; si la vecindad entera gana dinero, hay una meseta.

Ese mapa es el `optimizationProfile.bin` que cada `.sqx` del databank de salida se lleva dentro, y es lo único que `studies/breakage/spp/` sabe leer.

⚠ El SPP es `OptProfileSysParamPermutation`, no `SequentialOptimization`

Los dos viven **en el mismo bloque** de crosschecks, los dos hablan de permutar parámetros, y sólo uno es el SPP. Encender el otro costó 47 núcleos durante 91 minutos y no produjo perfil ninguno (medido el 2026-09-22, está en `knowhow/sqx-drive/spp-task-type.md`). El comando enciende el correcto y no te deja elegir: ésa es media razón de que exista.

Cuándo lo usas, y cuándo no

Úsalo cuando ya tienes un custom project con una población cribada dentro — lo que sobrevive al MC Retest— y quieres el mapa de parámetros de cada superviviente.

No lo uses sobre una población sin cribar. Un SPP son miles de backtests **por estrategia**: es con diferencia el paso más caro de la cadena, y sobre diez mil estrategias no termina nunca.

No lo uses para filtrar. Este comando apaga los cuatro chequeos de aceptación del propio SPP (`ProfitOptPct` y compañía) a propósito: si SQX borra lo que no le gusta, `sppUltra` está leyendo la superficie de una población ya seleccionada, sin saberlo.

Las dos tareas, y por qué no se comparan entre sí

tarea	ventana	qué es
`SPP IS`	`build` (2008–2017 en XAUUSD)	la vecindad en la muestra con la que se construyó
`SPP OOS`	`oos1` (2018–2022)	la misma vecindad leída fuera de muestra

Parecen un antes y un después y **no lo son**: medido el 2026-09-19 sobre `Strategy 17.9.39`, el SPP de IS y el de OOS de la misma estrategia, con ajustes idénticos, comparten **6 tuplas de ~11.600**. SQX no recorre la misma rejilla dos veces. Son dos lecturas del mismo entorno, y ése es exactamente el motivo de que exista la fábrica de variantes del paso 16.5: para comparar hay que fabricar las tuplas uno mismo.

Antes de empezar

- El proyecto tiene que existir y ser **custom** (`sqx.projects.builder`), clonado del donante congelado, que es el que trae las tareas `SPP IS` y `SPP OOS` con esos títulos. Una tarea que no esté se reporta con `⊘` y no se inventa.
- El `.cfx` **no puede estar abierto** por una instancia de SQX: la reescribe al salir y se lleva el cambio por delante sin decir nada (regla dura 4). El comando lo comprueba y se niega.
- Costes pactados: `python3 -m core.assets <SIMBOLO>` tiene que salir en verde (regla dura 5).

No toca SQX ni ningún worker: sólo lee y reescribe un fichero. Tarda menos de un segundo.

Cómo se ejecuta

```
python3 -m core.assets XAUUSD                       # preflight, BLOQUEANTE
python3 -m sqx.projects.spp XAUUSD \
    --cfx ~/Desktop/SQX_w2/user/projects/XAUUSD_estudio/project.cfx \
    --input "MCR 8 Stress"
```

flag	obligatorio	qué hace
`<SIMBOLO>`	sí	el activo, para sus costes y sus ventanas
`--cfx`	sí	el `project.cfx` del custom project
`--input`	sí	el databank que lee la **primera** tarea. No hay valor por defecto: son los supervivientes del paso anterior, y adivinarlo es correr el paso más caro sobre la población equivocada
`--json`	no	la misma información como JSON, para encadenar

La segunda tarea lee lo que escribió la primera. Con la aceptación apagada no se cae nadie, así que la población entera llega también a la lectura de fuera de muestra.

Qué produce

Reescribe **en el sitio** el `project.cfx` que le des: las dos tareas quedan activas, con el crosscheck encendido, la rejilla escrita, las condiciones apagadas y sus databanks encadenados. No escribe nada más y no crea copia de seguridad — el `.cfx` es del proyecto, y para volver atrás se clona otra vez del donante.

Cómo se lee el resultado

```
XAUUSD  XAUUSD  <- MCR 8 Stress
  ✓ SPP IS   build 2008.01.01→2017.12.31  ±35 % en 18 pasos, tope 15,000 permutaciones, precision 1
      MCR 8 Stress → SPP IS   19 condicion(es) y 2 chequeo(s) del SPP apagados
  ✓ SPP OOS  oos1 2018.01.01→2022.12.31  ±35 % en 18 pasos, tope 15,000 permutaciones, precision 1
      SPP IS → SPP OOS   19 condicion(es) y 2 chequeo(s) del SPP apagados

El perfil lo guarda cada .sqx del databank de salida: sacalo con sqx/export/export_spp.py y leelo
con studies/breakage/spp/.
⚠ Una SPP a la vez por instalacion: el perfil ocupa decenas de GB mientras se construye.
```

Qué mirar, línea a línea:

- **`±35 % en 18 pasos`** — cuánto se mueve cada parámetro y con qué finura. Un paso son ~4 %, que es el default del dueño. Los 12 pasos sobre ±30 que trae el donante son 5 % por paso y son gruesos.
- **`tope 15,000 permutaciones`** — ⚠ la línea más importante. El donante trae `1000000001`, que es el centinela de SQX para **exhaustivo**: todas las combinaciones de todos los parámetros. Eso no es una tarea larga, es una tarea sin final. El comando lo escribe siempre, nunca lo hereda.
- **`19 condicion(es) ... apagadas`** — si sale `0` en las dos, el proyecto ya venía con ellas apagadas; merece comprobarlo, porque lo normal en un clon del donante es 19.
- **`2 chequeo(s) del SPP apagados`** — los `Eval*Check` del propio crosscheck, que son su aceptación interna. Con ellos vivos, SQX borra las estrategias cuyo mapa no le gusta.
- **`precision 1`** — el backtest corre al timeframe de la tarea, no a 1 minuto. ⚠ Esto **diverge** de `precision.default: 2` de la doctrina, y es a propósito: son miles de backtests por estrategia y a 1 minuto no acaba. Es lo que llevan las tareas del maestro. Está anotado en `OPEN.md` como decisión pendiente del dueño.

- Un `⊘` en vez del `✓` significa que el proyecto no lleva esa tarea: se clonó sin ella.
- Un `⚠ esta tarea corre ademas: …` significa que ese `.cfx` no pasó por la doctrina y va a correr otros crosschecks a la vez. Se arregla clonando con `sqx.projects.builder`.

Un ejemplo completo

```
# 1. el proyecto ya existe, con su población cribada en "MCR 8 Stress"
python3 -m core.assets XAUUSD
python3 -m sqx.projects.spp XAUUSD --cfx ~/Desktop/SQX_w2/user/projects/XAUUSD_estudio/project.cfx \
    --input "MCR 8 Stress"

# 2. correrlo en el custodio, UNA sola cosa a la vez
bin/sqx-worker.sh --role custodian start
python3 -c "from core import worker; worker.call('-project action=start
name=XAUUSD_estudio','custodian')"
# ... termina ...
bin/sqx-worker.sh --role custodian stop

# 3. sacar los perfiles y leerlos (pasos 15→16)
python3 -m sqx.export.export_spp --project XAUUSD_estudio --databank "SPP IS" --role custodian
python3 -m studies.breakage.spp.report --project XAUUSD_estudio --databank SPP_IS
```

Qué NO te dice

- **No dice si la estrategia es buena.** Dice cómo se comporta su vecindad de parámetros. Una estrategia mala puede tener una meseta preciosa de resultados malos.
- **No compara IS con OOS.** Las dos rejillas no coinciden (6 tuplas de 11.600); quien compare columna con columna entre los dos databanks está comparando estrategias distintas.
- **No filtra nada**, por diseño. El veredicto lo da `studies/breakage/spp/` y se aplica con `/curate`.
- **El conteo `permutations` del perfil no es el número de filas.** Medido: un perfil que dice 2.533 exporta 3.940 filas. Para saber cuántas hay de verdad, `len(results)`.

Si algo falla

lo que sale	qué significa
`el custodian tiene este proyecto abierto…`	hay una instancia levantada sobre ese `.cfx`. Párala y repite; si no, SQX se come el cambio al salir
`⊘ SPP IS NO configurada — el proyecto no lleva la tarea SPP IS`	el clon se hizo sin esa tarea. Vuelve a clonar con `--tasks Retest` y el `--only` que la incluya
`XAUUSD: … sin valor pactado`	falta un coste en `assets/symbols/`. Pregúntale al dueño (regla dura 5)
el run termina en segundos y no hay perfil	señal clásica de que se corrió sobre una variante fabricada sin parámetros reales. Un SPP de verdad tarda media hora larga por estrategia y **no dice nada** mientras tanto: la señal honesta de que trabaja es la memoria de la JVM subiendo

8. SPP — sacar a Python todo lo que deja el Sys. Param Permutation

Qué pregunta responde

Cuando pasas una estrategia por el cross-check **Sys. Param Permutation** de StrategyQuant X, SQX mueve un poco cada parámetro de la estrategia, la vuelve a backtestear miles de veces y te enseña un panel con histogramas. Ese panel es lo único que ves dentro del programa, y no se puede exportar desde el menú.

Este comando saca **todo ese panel a CSV**: cuántas permutaciones se probaron, cuántas fueron rentables, la mediana de cada una de las 135 métricas de SQX, el valor original de la estrategia y el histograma completo de cada métrica, bin a bin. Y si SQX guardó el detalle, además **una fila por permutación** con sus parámetros y sus 152 estadísticos.

La pregunta de fondo es la del sobreajuste. Si tu estrategia gana 24.269 € y la permutación mediana gana 1.858 €, tu resultado **no es lo normal de esa familia de parámetros: es el pico**. Ese cociente —la columna `orig_over_median` — es el número que buscas.

Cuándo lo usas, y cuándo no

Úsalo cuando ya has corrido la tarea SPP dentro de SQX y las estrategias están volcadas en un databank (`SPP IS`, `SPP OOS`...).

Cuándo NO sirve:

- Si el databank no ha pasado por SPP, los `.sqx` no llevan perfil dentro y el comando no encuentra nada. No es un error: sencillamente esas estrategias no tienen ese dato.
- **No te da los trades de las permutaciones, y nunca podrá dártelos.** No es una limitación del exportador: es que SQX no los escribe. Cada permutación se guarda como su cadena de parámetros más un bloque `SQStats`, y ese bloque solo admite seis tipos de registro, todos numéricos —enteros, flotantes y longs—; no hay ninguna vía por la que se escriba una lista de órdenes. Las operaciones que sí tienes son las del backtest principal, y ésas se sacan con `python3 -m sqx.export.export_trades --project XAUUSD --databank "SPP IS" --symbol XAUUSD_DukasM1_Infinox`.
- **El detalle permutación a permutación solo existe si la casilla estaba quitada al correr el SPP.** Es *Settings → Performance → "Don't store data for 3D charts in Optimization profile"*. Con la casilla puesta SQX tira esos resultados al guardar y deja únicamente medianas e histogramas — y las estrategias guardadas así no lo recuperan aunque la quites después: hay que **volver a correr el SPP**. Con la casilla quitada el `.sqx` engorda mucho: los cinco de `SPP IS` pasaron de 125 KB a 2,2 MB cada uno.

Antes de empezar

- El databank tiene que estar **volcado a disco**. Si la carpeta está vacía, abre SQX, entra en el databank y deja que sincronice.
- **No hace falta arrancar nada.** El comando lee los `.sqx` directamente, no usa SQX ni el worker, y puedes lanzarlo con la GUI abierta sin ningún riesgo.

Cómo se ejecuta

```
python3 -m sqx.export.export_spp --project XAUUSD --databank "SPP IS"
```

flag	obligatorio	qué hace
`--project`	sí	nombre del proyecto tal y como aparece en SQX
`--databank`	sí	nombre del databank, con sus espacios y entre comillas si los tiene

Tarda un par de segundos por estrategia. No toca SQX.

```
$ python3 -m sqx.export.export_spp --project XAUUSD --databank "SPP IS"
5 profiles -> /home/sergioguslw/Desktop/AlgoData/raw/XAUUSD/SPP_IS/2026-09-10/spp
  runs.csv         5 rows
  metrics.csv      675 rows
  histograms.csv   13500 rows
  permutations.csv 21205 rows
  permutation_params.csv 213642 rows
```

Qué produce

En `~/Desktop/AlgoData/raw/<proyecto>/<databank>/<fecha>/spp/`:

archivo	qué es
`runs.parquet`	una fila por estrategia: permutaciones, cuántas ganaron y perdieron, beneficio medio y máximo, desviación típica y los parámetros que se movieron
`metrics.parquet`	una fila por estrategia y métrica: la mediana de las permutaciones, el valor original y su cociente
`histograms.parquet`	una fila por estrategia, métrica y bin: la frecuencia, y qué bin contiene la mediana y cuál el valor original
`spp.parquet`	**solo si SQX guardó el detalle** — **una fila por permutación**: `strategy`, `permutation`, una columna por parámetro que se movió (vacía en las estrategias que no mueven ese parámetro) y después los 152 estadísticos. La fila `permutation = -1` es la estrategia original, contra la que se comparan las demás. Desde el 23-09-2026 sustituye a `permutations.csv` + `permutation_params.csv`: 49 MB de CSV son 6 MB de parquet, y quien necesita cuatro columnas lee cuatro columnas
`manifest.json`	qué se leyó, cuántos perfiles había y con qué versión del código

Las carpetas van por fecha: una exportación nueva no borra la anterior.

Cómo se lee el resultado

```
metrics.csv  ·  Strategy 1.19.29  ·  4309 permutaciones

metric                 median          orig  orig/median
NetProfit             1544.68      24269.22        15.71
ProfitFactor             1.02          1.32         1.29
SharpeRatio              0.04          0.64        16.00
ReturnDDRatio            0.17          3.86        22.71
DrawdownPct             12.91          5.52         0.43
NumberOfTrades         754.00        763.00         1.01
WinningPct              49.09         60.16         1.23
RExpectancy              0.01          0.13        11.02
Stability                0.48          0.81         1.69
RSquared                 0.68          0.92         1.35
SQN                      0.07          0.77        11.00
CAGR                     0.15          2.20        14.67

histograms.csv  ·  NetProfit  ·  20 bins

 bin        edge   freq
   0   -91726.57      3
   1   -83065.79      9
   2   -75242.62     11   #
   3   -67184.05     12   #
   4   -59379.72     28   #
   5   -52520.72     21   #
   6   -43520.67     33   ##
   7   -35642.23     57   ###
   8   -27890.06     73   ####
   9   -20005.67    221   ###########
  10   -12177.81    534   ##########################
  11    -4371.00    626   ###############################
  12     3455.02    706   ################################### <- mediana
  13    11275.22    488   ########################
  14    19064.38    546   ###########################
  15    26881.34    551   ########################### <- original
  16    34665.00    290   ##############
  17    42304.19     94   #####
  18    47129.08      5
  19    50364.92      1
```

Arriba, `metrics.csv` para una estrategia. **La columna que decide es `orig_over_median`:**

- **Cerca de 1** — tu estrategia rinde como la permutación típica. El resultado no depende de haber acertado los parámetros exactos. Es lo que quieres ver.
- **Muy por encima de 1** — el resultado vive en un pico. En el ejemplo, `NetProfit` sale 13 veces la mediana y `ReturnDDRatio` 15 veces: mover los parámetros un punto destruye la estrategia.
- **Por debajo de 1 en una métrica de riesgo** (`DrawdownPct` sale 0,45) — el original sufre menos drawdown que la permutación mediana, que es la misma señal vista del revés.

Abajo, `histograms.csv` para `NetProfit`. Las 4.301 permutaciones repartidas en 20 bins, con la mediana en el bin 13 y el valor original en el 15. Lo que importa es **cuánta masa queda a la izquierda del cero**: aquí unas 1.300 de 4.301 permutaciones pierden dinero.

`runs.csv` resume lo mismo en una línea. `profitable_pct` es el porcentaje de permutaciones con beneficio; en las cinco estrategias de `SPP IS` va del 54 % al 92 %, y la de 92 % es la que menos depende de sus parámetros.

Y con el detalle guardado, lo que de verdad quieres

```
permutations.csv  ·  Strategy 1.19.29  ·  4309 permutaciones x 152 estadisticos

metric              original     mediana          p5         p95  percentil
NetProfit           24269.22     1544.68   -30311.12    30635.28      87.1%
ProfitFactor            1.32        1.02        0.72        1.52      86.9%
SharpeRatio             0.64        0.04       -0.69        0.83      87.3%
ReturnDDRatio           3.86        0.17       -0.88        6.28      88.2%
DrawdownPct             5.52       12.91        1.80       38.83      19.1%
NumberOfTrades        763.00      754.00       35.00     1768.00      51.1%
Stability               0.81        0.48       -0.81        0.85      90.0%
RSquared                0.92        0.68        0.01        0.95      86.7%

permutation_params.csv  ·  NetProfit mediano por valor de un parametro

   LWMAPeriod1      n  NetProfit med.
             8    325         291.06
             9    273       -3075.79
            10    616        2690.02
            11    325        4434.71
            12    319        2448.69
            13    270       -3600.57
            14    383        3292.84
            15    258       -2834.89
            16    465        5517.88
            17    461        6548.57
            18    328        -243.86
            19    287        -417.04
```

Arriba, lo que las medianas no te podían dar: **el percentil**. `NetProfit` original está en el percentil 87 de sus propias permutaciones, y `Stability` en el 90. Y los cuantiles: el 5 % peor de las permutaciones pierde 30.311 €. Con `metrics.csv` solo sabías la mediana; aquí tienes toda la distribución y puedes pedir el percentil que quieras.

Abajo, lo que **ninguna** parte del panel de SQX enseña: **qué parámetro es el que hace daño**. Agrupando las 4.309 permutaciones por el valor de `LWMAPeriod1`, el `NetProfit` mediano salta de −3.601 € a +6.549 € según el valor. Eso es una superficie de parámetros, y es lo que decide si la estrategia tiene una meseta donde vivir o un pico. Se hace en tres líneas:

```
w = q[q.strategy == "Strategy 1.19.29"].pivot(index="permutation", columns="parameter",
values="value")
j = w.join(p[p.permutation >= 0].set_index("permutation")["NetProfit"])
print(j.groupby("LWMAPeriod1").NetProfit.median())
```

Un ejemplo completo

```
$ python3 -m sqx.export.export_spp --project XAUUSD --databank "SPP IS"
5 profiles -> /home/sergioguslw/Desktop/AlgoData/raw/XAUUSD/SPP_IS/2026-09-10/spp
  runs.parquet     5 rows
  metrics.parquet  675 rows
  histograms.parquet 13500 rows
  spp.parquet      21205 rows
```

Y en Python:

```
import pandas as pd
from pathlib import Path

d = Path.home() / "Desktop/AlgoData/raw/XAUUSD/SPP_IS/2026-09-10/spp"
m = pd.read_parquet(d / "metrics.parquet")

frag = m[m.metric == "NetProfit"].set_index("strategy")["orig_over_median"]
print(frag.sort_values(ascending=False))
```

Para leer un perfil suelto, sin exportar nada:

```
from pathlib import Path
from core import optprofile

p = optprofile.read(Path.home() / "Desktop/SQX/user/projects/XAUUSD/databanks/SPP IS/Strategy
1.19.29.sqx")
print(p["permutations"], p["profitable_pct"], p["params"])
print(p["medians"]["NetProfit"], p["orig"]["NetProfit"])

if p["permutation_results"]:                        # la casilla estaba quitada
    print(p["results"][0]["params"])
    print(p["results"][0]["stats"]["NetProfit"])
```

Qué NO te dice

- **No es una prueba fuera de muestra.** Todas las permutaciones se corren sobre el mismo tramo de datos que la estrategia original. Un SPP impecable sigue siendo compatible con una estrategia que no funcione en el futuro; lo que descarta es una cosa distinta —que el resultado dependa de haber clavado los parámetros—, no que el edge sea real.
- **Los histogramas vienen ya agregados por SQX**, en 20 bins que él eligió. Si el databank no guardó el detalle, ésa es toda la distribución que hay: no se puede pedir otro percentil ni cruzar dos métricas permutación a permutación. Con el detalle guardado, `spp.parquet` deja hacer las dos cosas y los histogramas sobran.
- **En `spp.parquet`, 34 de los 152 estadísticos salen como `stat:<tipo>:<índice>`.** SQX los guarda por su posición en un array, sin nombre. Los 118 nombrados se calibraron cruzando el resultado original de 34 perfiles contra la tabla de medianas del mismo fichero, que sí lleva nombre; los 34 restantes valen 0 en todas las estrategias, que es justo por lo que no hubo con qué distinguirlos. El inventario completo, campo a campo, está en el capítulo 9.

- **28 de las 135 métricas de `metrics.parquet` salen como `id:<número>`.** SQX guarda las métricas bajo un hash de su nombre que ya no se puede deshacer, así que los nombres se reconstruyeron comparando valores contra una exportación de databank de 50 estrategias. Esas 28 valen exactamente 0 en todas las estrategias de esta instalación, así que no había con qué distinguirlas — y por eso mismo tampoco aportan nada. Otras seis llevan interrogación (`Exposure?` / `ExposurePosition?`, `Outlier?` / `Outlier2?`, `CalmarRatio?` / `AnnualPctReturnDDRatio?`): son tres parejas cuyos dos miembros valen lo mismo en las 225 estrategias con perfil de esta máquina, así que se sabe **de qué pareja es cada columna, pero no cuál de las dos**. Las 101 restantes están verificadas una a una.

Si algo falla

- **`0 profiles`** — el databank no ha pasado por SPP, o los `.sqx` no están en disco. Comprueba la carpeta `user/projects/<proyecto>/databanks/<databank>/`.
- **`KeyError: 'optimizationProfile.bin'`** no puede salir: el comando salta los `.sqx` que no lo llevan. Si sale otra cosa al leer un `.sqx`, es que ese archivo está corrupto; el traceback dice cuál.

9. Diccionario — todo lo que se puede extraer, campo por campo

Esta página no explica cómo se corre nada: eso está en el capítulo 8. Aquí está el **inventario completo** de lo que sale de una estrategia que ha pasado por el Sys. Param Permutation, con el nombre exacto de cada campo y qué es. Es la página que se tiene abierta al lado mientras se escribe el `pandas`.

Todo lo que aparece aquí se lee del `.sqx` directamente. No hace falta SQX abierto ni el worker.

De dónde sale cada cosa

fichero	una fila por	lo llena
`runs.parquet`	estrategia	el resumen de la tirada de permutaciones
`metrics.parquet`	estrategia × métrica	la tabla mediana / valor original del panel
`histograms.parquet`	estrategia × métrica × bin	los histogramas que dibuja el panel
`spp.parquet`	estrategia × permutación	**solo si la casilla 3D estaba quitada**: los parámetros en columnas y después los estadísticos. Hasta el 23-09-2026 eran dos CSV, `permutations.csv` (estadísticos) y `permutation_params.csv` (parámetros en forma larga); las secciones 4 y 5 describen las columnas, que son las mismas
`trades.parquet`	operación	el backtest principal, vía `export_trades.py`
la librería de barras (`03-datos-costes-y-registro.pdf › 13-barras`)	barra	las velas sobre las que operó

---

1. `runs.parquet` — la tirada entera en una línea

campo	qué es
`strategy`	nombre de la estrategia, tal y como aparece en el databank
`permutations`	cuántas permutaciones se probaron
`profitable`	cuántas acabaron con beneficio positivo
`losing`	cuántas acabaron en pérdida
`zero`	cuántas acabaron exactamente en cero, normalmente porque no llegaron a operar
`profitable_pct`	`profitable` en porcentaje. **La cifra de cabecera del SPP**: cuánto del entorno de parámetros gana
`avg_profit`	beneficio medio de todas las permutaciones, en divisa de la cuenta
`top_profit`	el beneficio de la mejor permutación
`stdev`	desviación típica del beneficio entre permutaciones. Mide lo mucho que se mueve el resultado al mover los parámetros
`uniform_changes`	contador interno de SQX sobre lo uniforme que es la distribución. No está documentado qué escala usa
`parameters`	los nombres de los parámetros que se permutaron, separados por espacio

Hay dos campos más que el lector de Python devuelve y que no se escriben al CSV porque no son tabulares: `last_count` (repite `permutations`) y `stdev_computed` (siempre 0 en esta instalación).

2. `metrics.parquet` — la tabla del panel

campo	qué es
`strategy`	nombre de la estrategia
`metric`	nombre de la métrica; los 135 posibles están en la tabla grande de más abajo
`median`	el valor **mediano** de esa métrica entre todas las permutaciones
`orig`	el valor de la estrategia original, la que tú tienes guardada
`orig_over_median`	`orig / median`. **La columna que decide.** Cerca de 1 el resultado no depende de los parámetros; muy por encima, vive en un pico

3. `histograms.parquet` — la distribución tal y como la agrupó SQX

campo	qué es
`strategy`	nombre de la estrategia
`metric`	métrica a la que pertenece el histograma
`bin`	número de bin, de 0 a 19. Siempre son 20
`edge`	etiqueta del borde inferior del bin. **Puede venir vacía**: SQX se salta etiquetas para que quepan en el eje
`frequency`	cuántas permutaciones cayeron en ese bin
`is_median`	`True` en el bin donde cae la mediana
`is_orig`	`True` en el bin donde cae el valor original

4. `spp.parquet`, los estadísticos — una fila por permutación

campo	qué es
`strategy`	nombre de la estrategia
`permutation`	número de permutación desde 0. **La fila `-1` es la estrategia original**, para comparar contra ella sin buscarla aparte
los otros 152	los estadísticos, uno por columna. Los nombrados son los de la tabla grande; los demás salen como `stat:<tipo>:<índice>`

5. `spp.parquet`, los parámetros — qué valores le tocaron a cada permutación (una columna por parámetro; antes, forma larga con `parameter` y `value`)

campo	qué es
`strategy`	nombre de la estrategia
`permutation`	mismo número que en `permutations.csv`; se cruzan por `strategy` + `permutation`
`parameter`	nombre del parámetro, tal y como lo llama la estrategia (`LWMAPeriod1`, `ExitAfterBars1`...)
`value`	el valor que SQX le puso en esa permutación

Los nombres de parámetro **cambian de una estrategia a otra**: son los de sus propios bloques. Por eso van en formato largo y no en columnas.

---

6. Las 135 métricas

Esto es el catálogo entero de columnas de databank de tu instalación, que es exactamente lo que el SPP guarda por métrica. Los nombres cortos son los que usan `metrics.csv` y `permutations.csv`; el nombre en SQX es el que ves en la GUI.

- **unidad** — `€` divisa de la cuenta · `%` porcentaje · `pips` · `ratio` · `entero` · `texto` · `fecha` · `–` no declarada en el código de la columna.
- **mejor** — hacia dónde considera SQX que es mejor: `↑` más alto, `↓` más bajo, `≈` ni una cosa ni otra, `–` no declarado.
- **por permutación** — si además aparece en `permutations.csv`, es decir si se puede mirar permutación a permutación y no solo como mediana.

campo	nombre en SQX	qué es	unidad	mejor	por permutación
`AHPR`	AHPR	Rendimiento medio por periodo de tenencia (Arithmetic Holding Period Return), calculado sobre los retornos anuales.	ratio	↓	sí
`ActualDD`	Actual Drawdown	Drawdown que había abierto durante la ÚLTIMA operación del backtest. No es el máximo, es el del final.	—	—	sí
`ActualDrawdownPct`	Actual Drawdown / Max DD	El drawdown de la última operación dividido por el drawdown máximo. Cerca de 1 significa que el backtest termina en su peor momento.	—	—	sí
`AddMarketsNetProfitMedian`	AddMarketsNetProfitMedian	Mediana del beneficio neto en los mercados adicionales del retest cruzado. Vale 0 si la estrategia no se retesteó en otros mercados.	—	—	sí
`AddMarketsProfitFactorMedian`	AddMarketsProfitFactorMedian	Mediana del profit factor en los mercados adicionales del retest cruzado.	—	—	sí
`AddMarketsRetDDMedian`	AddMarketsRetDDMedian	Mediana del Ret/DD en los mercados adicionales del retest cruzado.	—	—	sí
`AddMarketsSharpeRatioMedian`	AddMarketsSharpeRatioMedian	Mediana del Sharpe en los mercados adicionales del retest cruzado.	—	—	sí
`AmbiguousTrades`	Ambiguous Trades	Operaciones que abren y cierran en la misma barra. Son ambiguas porque el backtest no puede saber qué pasó primero dentro de la barra.	entero	↓	sí
`AmbiguousTradesPct`	Ambiguous Trades %	Las anteriores como porcentaje del total de operaciones. Es la	%	↓	sí

campo	nombre en SQX	qué es	unidad	mejor	por permutación
		métrica de fiabilidad del backtest, no de rentabilidad.			
`AnnualPctReturn`	Annual % Return	Rentabilidad porcentual anualizada sobre los años de datos.	%	↑	sí
`AnnualPctReturnDDRatio`	CAGR/Max DD %	CAGR dividido por el drawdown máximo en porcentaje.	ratio	↑	sí
`AvgAbsTrade`	Avg. Abs Trade	Beneficio absoluto medio por operación, contando las perdedoras en valor absoluto.	€	↑	sí
`AvgBarsInTrade`	Avg. Bars in Trade	Barras que dura una operación de media.	ratio	↓	sí
`AvgBarsLoss`	Avg. Bars Loss	Barras que dura de media una operación perdedora.	ratio	↓	sí
`AvgBarsWin`	Avg. Bars Win	Barras que dura de media una operación ganadora.	ratio	↑	sí
`AvgConsecLosses`	Avg Consec. Losses	Longitud media de las rachas de pérdidas.	ratio	↓	sí
`AvgConsecWins`	Avg Consec. Wins	Longitud media de las rachas de ganancias.	ratio	↑	sí
`AvgDrawdown`	Avg. Drawdown	Drawdown medio de todos los que hubo, no solo el máximo.	€	↓	sí
`AvgLoss`	Avg. Loss	Pérdida media de las operaciones perdedoras.	€	↓	sí
`AvgParametersStability`	Avg. Parameters Stability	Estabilidad media de los parámetros en el Walk Forward Matrix. Solo tiene valor si se corrió WF.	ratio	↑	no
`AvgPctDrawdown`	Avg. % Drawdown	Drawdown medio expresado en porcentaje.	%	↓	sí
`AvgPctProfitPerYear`	Avg. % Profit Per Year	Beneficio porcentual medio por año.	%	↑	sí
`AvgProfitPerDay`	Avg. Profit Per Day	Beneficio medio por día de trading.	€	↑	sí
`AvgProfitPerMonth`	Avg. Profit Per Month	Beneficio medio por mes.	€	↑	sí

campo	nombre en SQX	qué es	unidad	mejor	por permutación
`AvgProfitPerYear`	Avg. Profit Per Year	Beneficio medio por año.	€	↑	sí
`AvgTrStddevRatio`	Avg. trade / StdDev ratio	Operación media dividida por la desviación típica de las operaciones. Es el Sharpe por operación, sin anualizar.	—	—	sí
`AvgTrade`	Avg. Trade	Beneficio medio por operación (expectativa en dinero).	€	↑	sí
`AvgTradesPerDay`	Avg. Trades Per Day	Operaciones por día.	ratio	↓	sí
`AvgTradesPerMonth`	Avg. Trades Per Month	Operaciones por mes.	ratio	↓	sí
`AvgTradesPerYear`	Avg. Trades Per Year	Operaciones por año.	ratio	↓	sí
`AvgWin`	Avg. Win	Ganancia media de las operaciones ganadoras.	€	↑	sí
`BacktestDuration`	Backtest Duration (s)	Segundos que tardó el backtest en correr. Es un dato de la máquina, no de la estrategia.	texto	↓	no
`BestWF`	Best WF	Mejor resultado del Walk Forward. Vacío si no se corrió WF.	texto	↑	sí
`BiggestMAE`	Biggest MAE	La peor excursión adversa de todas las operaciones: cuánto llegó a estar en contra la operación que más sufrió.	€	↑	sí
`CAGR`	CAGR	Tasa de crecimiento anual compuesta.	%	↑	sí
`CalmarRatio`	CalmarRatio	Rentabilidad anualizada dividida por el drawdown máximo.	ratio	↑	sí
`Commission`	Commission/Swap ($)	Comisiones y swap pagados en total, en divisa de la cuenta.	€	↓	sí
`Complexity`	Complexity	Medida de complejidad de la estrategia que calcula SQX a partir de su estructura.	entero	↓	sí
`DateGenerated`	Date generated	Fecha en que SQX generó la estrategia.	fecha	↓	no

campo	nombre en SQX	qué es	unidad	mejor	por permutación
`DateLastModified`	Date last modified	Fecha del último cambio o retest de la estrategia.	fecha	↓	no
`DegreesOfFreedom`	Degrees of freedom	Grados de libertad: operaciones menos parámetros. Cuanto más bajo, más fácil es que el resultado sea casualidad.	entero	↓	sí
`DoFRatio`	DoF Ratio	Operaciones divididas por número de parámetros optimizados. Cuántos datos hay por cada grado de libertad.	ratio	↑	sí
`Drawdown`	Drawdown	Máxima caída desde un máximo de la curva de balance, en divisa de la cuenta.	€	↓	sí
`DrawdownPct`	Max DD %	El drawdown máximo en porcentaje.	%	↓	sí
`DrawdownPctOnInitial`	DD% of Initial	El drawdown máximo como porcentaje del depósito inicial, no del pico.	—	—	sí
`DrawdownPips`	Max DD pips	El drawdown máximo medido en pips.	pips	↓	sí
`EdgeDecayRatio`	Edge Ratio Decay	Puntuación 0-100 de calidad fuera de muestra construida sobre cuatro pilares. Columna propia, no estándar de SQX.	ratio	↑	sí
`EdgeRatioInPips`	Edge Ratio	Edge ratio en pips: relación entre excursión favorable y adversa. La definición exacta no viene documentada en esta instalación.	—	—	sí
`Efficiency`	Efficiency	Beneficio neto dividido por beneficio bruto: qué parte de lo que ganó se quedó después de las pérdidas.	—	—	sí
`EntryIndicators`	Entry indicators	Lista de indicadores que usan las condiciones de entrada. Texto.	texto	↑	no

campo	nombre en SQX	qué es	unidad	mejor	por permutación
`EquityAngle`	EquityAngle	Ángulo de la recta de regresión de la curva de equity.	ratio	↑	sí
`EquitySlope`	EquitySlope	Lo recta que es la curva de equity multiplicado por lo empinada que es.	ratio	↑	sí
`ExitIndicators`	Exit indicators	Lista de indicadores que usan las condiciones de salida. Texto.	texto	↑	no
`ExitQuality`	Exit quality	Beneficio total dividido por el MFE total: cuánto del movimiento favorable disponible llegó a capturar.	ratio	↑	sí
`Expectancy`	Expectancy	Expectativa por operación.	ratio	↑	sí
`Exposure`	Exposure	Barras con posición abierta dividido por barras totales de la muestra.	%	↓	sí
`ExposurePosition`	Exposure Position	Lo mismo que Exposure, en la versión que usa el motor Stockpicker considerando el número de posiciones abiertas.	%	↓	sí
`FiltersResult`	Filters result	PASSED o FAILED según los filtros de aceptación del databank. Texto.	texto	↑	no
`Fitness`	Fitness	Valor de fitness con el que el algoritmo genético ordenó la estrategia.	ratio	↑	sí
`GiniOOS`	Gini Coeff	Coeficiente de Gini del P&L por operación: 0 es que todas aportan lo mismo, 1 es que una sola se lo lleva todo. Columna propia.	ratio	↓	sí
`GrossLoss`	Gross loss	Suma de todas las operaciones perdedoras.	€	↓	sí
`GrossProfit`	Gross profit	Suma de todas las operaciones ganadoras.	€	↑	sí

campo	nombre en SQX	qué es	unidad	mejor	por permutación
`InitialDeposit`	Initial deposit	Depósito inicial del backtest.	ratio	↑	no
`KellyFormula`	Kelly formula	Fracción de Kelly: qué porcentaje del capital sugeriría arriesgar la fórmula con este win rate y payout.	—	—	sí
`LongestTrade`	Longest trade (days)	Duración en días de la operación más larga.	entero	↓	sí
`MagicNumber`	Magic number	Identificador que SQX pone a la estrategia para el broker. No es una métrica.	texto	↓	no
`MaxConsecLosses`	Max Consec. Losses	Racha más larga de pérdidas seguidas.	entero	↓	sí
`MaxConsecWins`	Max Consec. Wins	Racha más larga de ganancias seguidas.	entero	↑	sí
`MaxIntradayDrawdown`	Max Intraday Drawdown	Drawdown máximo mirando también dentro del día, no solo balances de cierre.	€	↓	sí
`MaxLoss`	Max Loss	La peor operación individual.	€	↓	sí
`MaxNewHighDuration`	Max Drawdown Duration	Días que tardó como mucho en hacer un nuevo máximo de equity. Es la duración del drawdown más largo.	entero	↓	sí
`MaxProfit`	Max Profit	La mejor operación individual.	€	↑	sí
`MaxTSIntradayDrawdown`	Max TS Intraday Drawdown	Drawdown intradía máximo en la versión TradeStation.	€	↑	sí
`MiniEquityChart`	Mini equity chart	La miniatura de la curva de equity que pinta la GUI. No es un número.	texto	↓	no
`NSymmetry`	NSymmetry	Simetría de beneficio entre largos y cortos, con la variante de que si un lado gana y el otro pierde devuelve −1 en vez de 0.	%	↑	no
`NetProfit`	Net profit	Beneficio neto total en divisa de la cuenta. Es la métrica de referencia del SPP.	€	↑	sí

campo	nombre en SQX	qué es	unidad	mejor	por permutación
`NetProfitInPct`	Net profit in %	El beneficio neto en porcentaje.	%	↑	sí
`NetProfitInPips`	Net profit in pips	El beneficio neto en pips.	pips	↑	sí
`Note`	Note	Nota escrita a mano en la estrategia. Texto.	texto	↓	no
`NumberOfCanceled`	# of canceled	Órdenes pendientes que se cancelaron sin llegar a ejecutarse.	entero	≈	no
`NumberOfLosses`	# of losses	Número de operaciones perdedoras.	entero	↓	sí
`NumberOfProfits`	# of profits	Número de operaciones ganadoras.	entero	↑	sí
`NumberOfTrades`	# of trades	Número total de operaciones. En el SPP es la métrica de control: si una permutación baja de golpe, es que la regla dejó de dispararse.	entero	≈	sí
`OpenDrawdown`	Open Drawdown	Drawdown calculado sobre el balance abierto más el MAE, es decir contando lo que se sufrió sin cerrar.	€	↓	sí
`OpenDrawdownPct`	Open Drawdown %	El anterior en porcentaje.	%	↓	sí
`Outlier`	Outlier1	Peso de la operación atípica, SIN filtrar operaciones con el mismo P&L.	—	—	sí
`Outlier2`	Outlier1	Peso de la operación atípica, filtrando las operaciones con el mismo P&L.	—	—	sí
`ParameterCount`	Param Count	Parámetros realmente buscados: periodos, constantes, niveles de entrada y salidas usadas. Excluye MagicNumber, variables de señal, Shift y salidas sin usar. Columna propia.	entero	↓	sí
`Parameters`	Parameters	Volcado en texto de los parámetros de la	texto	↑	sí

campo	nombre en SQX	qué es	unidad	mejor	por permutación
		estrategia.			
`PayoutRatio`	Payout ratio	Ganancia media dividida por pérdida media.	ratio	↑	sí
`PriceIndicators`	Price indicators	Indicadores usados en los niveles de precio de entrada (stop o limit). Texto.	texto	↑	no
`ProbSharpeRatio`	PSR	Sharpe probabilístico (0-1): probabilidad de que el Sharpe verdadero sea mayor que cero. Columna propia.	ratio	↑	sí
`ProfitFactor`	Profit factor	Beneficio bruto dividido por pérdida bruta.	ratio	↑	sí
`ProfitableMonths`	Profitable Months	Meses cerrados en positivo.	entero	↑	sí
`ProfitableMonthsPct`	% Profitable Months	Porcentaje de meses en positivo.	%	↑	sí
`RExpectancy`	R Expectancy	Expectativa medida en R, es decir en múltiplos del riesgo por operación.	ratio	↑	sí
`RExpectancyScore`	R Expectancy Score	R Expectancy ponderada por el número de operaciones, para que una expectativa alta con pocas operaciones no gane.	ratio	↑	sí
`RSquared`	RSquared	R^2 de la regresión sobre la curva de equity: lo recta que es.	ratio	↑	sí
`RecoveryFactor`	RecoveryFactor	Beneficio neto dividido por el drawdown máximo.	ratio	↑	sí
`ResultsName`	Results Name	Nombre del resultado. Texto.	texto	↑	no
`ReturnDDRatio`	Ret/DD Ratio	Retorno dividido por drawdown.	ratio	↑	sí
`ReturnOpenDDRatio`	Ret/OpenDD Ratio	Retorno dividido por el drawdown abierto.	ratio	↑	sí
`SQN`	SQN	System Quality Number de Van Tharp.	ratio	↑	sí

campo	nombre en SQX	qué es	unidad	mejor	por permutación
`SQNScore`	SQN Score	SQN ponderado por el número de operaciones.	ratio	↑	sí
`SharpeRatio`	Sharpe Ratio	Ratio de Sharpe anualizado.	ratio	↑	sí
`SlippageInMoney`	Slippage ($)	Deslizamiento total pagado, en divisa de la cuenta.	€	↑	no
`SlopeRatio`	Slope Ratio	Pendiente fuera de muestra dividida por la pendiente en muestra (beneficio medio por operación OOS frente a IS). Columna propia.	ratio	↑	sí
`SortinoRatio`	Sortino Ratio	Ratio de Sortino: como el Sharpe pero castigando solo la volatilidad a la baja.	—	—	sí
`Stability`	Stability	Lo recta Y lo empinada que es la curva de equity, en una sola cifra.	ratio	↑	sí
`StabilitySQ3`	Stability SQ3	La misma estabilidad calculada como la calculaba StrategyQuant 3.	ratio	↑	sí
`Stagnation`	Stagnation	Días que la equity pasó estancada sin hacer nuevo máximo.	entero	↓	sí
`StagnationPct`	% Stagnation	El estancamiento como porcentaje de los días totales.	%	↓	sí
`StandardDev`	StandardDev	Desviación típica del P&L de las operaciones.	ratio	↓	sí
`Symbol`	Symbol	Símbolo sobre el que se hizo el test. Texto.	texto	↓	no
`Symmetry`	Symmetry	Simetría del beneficio entre el lado largo y el corto.	%	↑	no
`TRLRatio`	TRL Ratio	Operaciones reales divididas por el mínimo de operaciones necesarias para afirmar con 95 % de confianza que el Sharpe es mayor que cero. Columna propia.	ratio	↑	sí

campo	nombre en SQX	qué es	unidad	mejor	por permutación
`TSIndex`	TS Index	Índice TradeStation.	ratio	↑	sí
`TSWinLossRatio`	TS Win/Loss ratio	Ratio ganadoras/perdedoras en la definición de TradeStation.	ratio	↑	sí
`TimeFrame`	TimeFrame	Timeframe del test. Texto.	texto	↑	no
`TotalDataDays`	Total Data Days	Días que abarcan los datos.	entero	↓	sí
`TotalDataMonths`	Total Data Months	Meses que abarcan los datos.	entero	↓	sí
`TotalDataYears`	Total Data Years	Años que abarcan los datos.	entero	↓	sí
`TotalMFE`	Total MFE	Suma del MFE de todas las operaciones: todo el movimiento favorable que hubo disponible.	€	↓	sí
`TotalTradingDays`	Total Trading Days	Días en los que hubo actividad.	entero	↓	sí
`TotalTradingMonths`	Total Trading Months	Meses en los que hubo actividad.	entero	↓	sí
`TotalTradingYears`	Total Trading Years	Años en los que hubo actividad.	entero	↓	sí
`TradesSymmetry`	Trades Symmetry	Simetría del NÚMERO de operaciones entre largos y cortos.	%	↑	no
`UlcerIndex`	Ulcer Index %	Ulcer Index: mide la profundidad y la duración de los drawdowns a la vez.	—	—	sí
`UlcerPerformanceIndex`	Ulcer Performance Index	Ulcer Performance Index anualizado: retorno por unidad de Ulcer Index.	ratio	↑	sí
`WinLossRatio`	Win/Loss ratio	Ganadoras dividido por perdedoras.	ratio	↑	sí
`WinningPct`	Winning Percent	Porcentaje de operaciones ganadoras.	%	↑	sí
`WorstParametersStability`	Worst Parameters Stability	La peor estabilidad de parámetros de todos los Walk Forward. Solo tiene valor si se corrió WF.	ratio	↑	no
`WorstYearProfit`	Worst Year Profit	Beneficio del peor año.	€	↓	sí

campo	nombre en SQX	qué es	unidad	mejor	por permutación
`ZProbability`	ZProbability	Probabilidad asociada al Z-Score: si las rachas son o no compatibles con el azar.	ratio	↑	sí
`ZScore`	ZScore	Z-Score de las rachas: mide si ganancias y pérdidas se agrupan más de lo que cabría esperar por azar.	ratio	↑	sí

> **Aviso sobre `Outlier` y `Outlier2`:** SQX exporta las dos con la misma etiqueta, `Outlier1`. No es un fallo del exportador, es lo que trae la columna. Distínguelas por el nombre corto, nunca por la etiqueta.

Las 21 que no aparecen por permutación

`AvgParametersStability`, `BacktestDuration`, `DateGenerated`, `DateLastModified`, `EntryIndicators`, `ExitIndicators`, `FiltersResult`, `InitialDeposit`, `MagicNumber`, `MiniEquityChart`, `Note`, `PriceIndicators`, `ResultsName`, `Symbol`, `TimeFrame`, `WorstParametersStability` son metadatos o texto: describen la estrategia, no su resultado, así que no cambian de una permutación a otra y SQX no los repite.

`NSymmetry`, `Symmetry`, `TradesSymmetry`, `NumberOfCanceled` y `SlippageInMoney` sí son numéricas y sí tienen su hueco guardado — pero valen 0 en todas las estrategias de esta instalación, y por eso quedaron entre las que no se pudieron nombrar (siguiente apartado). Están, no se sabe en qué hueco.

---

7. Lo que solo existe por permutación

Cuatro estadísticos aparecen en `permutations.csv` y no son columna de databank, así que no salen en `metrics.csv`:

campo	qué es
`DataLength`	número de barras que consumió el backtest de esa permutación
`MaxNewHighDurationFrom`	inicio del drawdown más largo, en **epoch de milisegundos** (`pd.to_datetime(v, unit="ms")`)
`MaxNewHighDurationTo`	fin de ese mismo drawdown, mismo formato
`MaxNewHighDurationPct`	ese drawdown como porcentaje del periodo total

Y dos más llevan interrogación — `AnnualPctReturnDDRatio?` y `CalmarRatio?` — por el motivo del apartado siguiente.

---

8. Lo que no tiene nombre, y por qué

SQX no guarda los nombres: guarda un hash del nombre de la clase (en la tabla de métricas) o la posición en un array (en los estadísticos por permutación). El hash es `SQUtils.betterHashCode`, que vive en un jar embebido en el ejecutable y no está en disco, así que no se puede deshacer.

Los nombres de esta página se reconstruyeron **cruzando valores**: exportando las 135 columnas de un databank para 50 estrategias y viendo qué valor coincidía con cuál. Lo que no se pudo distinguir es lo que vale lo mismo en todas partes.

grupo	cuántos	qué son
verificados uno a uno	101 de 135 métricas	nombre seguro
identificados hasta la pareja	6 métricas	`Exposure?` / `ExposurePosition?`, `Outlier?` / `Outlier2?`, `CalmarRatio?` / `AnnualPctReturnDDRatio?`. Sabes de qué pareja es la columna, no cuál de las dos. Sus dos miembros valen lo mismo en los 225 perfiles de esta máquina
sin nombre, en `metrics.csv`	28 métricas	salen como `id:<número>`. **Valen 0 en todas las estrategias de esta instalación** — que es justo por lo que no hubo con qué distinguirlas, y también por lo que no aportan nada
sin nombre, en `permutations.csv`	34 estadísticos	salen como `stat:<tipo>:<índice>`, mismo motivo. `f` es un flotante, `i` un entero, `l` un long

Los identificadores concretos, por si algún día aparecen con valor distinto de cero y se pueden nombrar:

Métricas sin nombre (`metrics.csv`)

`id:-1032057831` `id:-1401870665` `id:-1558884181` `id:-1679613218`
`id:-1783525730` `id:-1792934761` `id:-1832811737` `id:-184077012`
`id:-1925735574` `id:-2028835157` `id:-2090795189` `id:-2100959438`
`id:-222360201` `id:-301431951` `id:-652428356` `id:-833100900`
`id:1041921791` `id:1775062197` `id:1876603141` `id:1945540697`
`id:195695425` `id:2059656148` `id:2133209636` `id:40355801`
`id:437767648` `id:500275187` `id:568822547` `id:982587326`

Estadísticos sin nombre (`permutations.csv`)

`stat:f:36` `stat:f:47` `stat:f:50` `stat:f:53` `stat:f:54` `stat:f:55`
`stat:f:58` `stat:f:6` `stat:f:62` `stat:f:63` `stat:f:64` `stat:f:65`
`stat:f:66` `stat:f:67` `stat:f:68` `stat:f:69` `stat:f:70` `stat:f:71`
`stat:f:72` `stat:f:73` `stat:f:74` `stat:f:75` `stat:f:76` `stat:f:77`
`stat:f:78` `stat:f:83` `stat:f:84` `stat:f:88` `stat:f:89` `stat:i:7`
`stat:l:0` `stat:l:1` `stat:l:2` `stat:l:3`

---

9. Lo que se extrae de la misma estrategia sin ser del SPP

`trades/<estrategia>.csv` — las operaciones del backtest principal

Son las que sí existen. 16 columnas, tal y como las escribe `orderstocsv`:

campo	qué es
`Ticket`	número de operación, empieza en 1
`Symbol`	símbolo. Si el fichero trae varios mercados, **esta columna es el separador**
`Type`	`Buy` o `Sell`
`Open time`	fecha y hora de entrada
`Open price`	precio de entrada
`Size`	tamaño en lotes. **Varía operación a operación** porque el sizing es por riesgo
`Close time`	fecha y hora de salida
`Close price`	precio de salida
`Profit/Loss`	resultado en divisa de la cuenta, ya con costes dentro
`Balance`	balance acumulado después de esta operación
`Sample type`	`IST` en muestra, `OOS1` fuera. **No siempre está bien**: depende de cómo se retesteó la estrategia por última vez
`Close type`	por qué se cerró: `Exit Signal`, `Exit After X Bars`, `End Of Friday (Time)`, `EndTest`...
`MAE ($)`	máxima excursión adversa, **en divisa de la cuenta, no en puntos**
`MFE ($)`	máxima excursión favorable, ídem
`Time in trade`	duración en texto (`2h 0m`)
`Comment`	comentario de la orden

La última fila puede ser una orden pendiente que nunca se ejecutó (`Close type=EndTest`, con precio de cierre vacío). Se tira.

`bars/bars_<TF>.csv` — las velas

`Date`, `Time`, `Open`, `High`, `Low`, `Close`, `Volume`. Es el fichero contra el que se reconcilian los precios de las operaciones.

Y del `.sqx`, sin exportar nada

qué	cómo
las métricas guardadas por muestra (IS / OOS / periodo completo)	`core.sqxstats.stats()`
la curva de equity diaria, un punto por día natural	`core.sqxstats.equity()`
el perfil SPP entero	`core.optprofile.read()`
símbolo, timeframe, hash de identidad, parámetros	`core.sqxfile`

---

10. Lo que no existe, por mucho que se busque

Los trades de las permutaciones. No es que falte el comando. Cada permutación se guarda como su cadena de parámetros más un bloque `SQStats`, y ese bloque solo admite seis tipos de registro — entero, long y flotante, cada uno con índice o con nombre — y nada más. La clase tiene un campo para objetos, pero el serializador no lo escribe nunca. No hay ninguna ruta por la que una lista de órdenes llegue al fichero.

La equity de cada permutación. Mismo motivo: `dailyEquity.bin` solo existe para el resultado principal.

Un percentil distinto de la mediana, si la casilla 3D estaba puesta. Con el detalle tirado solo quedan medianas e histogramas de 20 bins. Con el detalle guardado se puede pedir cualquier cosa.

15. SPP Ultra — ¿merece esta estrategia las 5.000 variantes?

Qué pregunta responde

Cuando pasas una estrategia por el cross-check **Sys. Param Permutation**, SQX prueba miles de combinaciones de sus parámetros y te enseña unos histogramas. Este comando lee esa tirada entera y contesta tres cosas que la pantalla de SQX no dice:

1. **Qué parámetros mueven de verdad el resultado** y cuáles están muertos.
2. **Dónde está la meseta** de cada parámetro — la zona ancha donde funciona, que no es lo mismo que el punto donde funciona mejor.
3. **Si toda la familia es ruido.** Si pruebas 3.000 combinaciones de algo que no sirve para nada, la mejor de las 3.000 va a parecer buena igualmente. Esto calcula cuánto de buena parecería por pura suerte, y compara.

Y deja preparado el **diseño de las 5.000 variantes** para la siguiente etapa.

Cuándo lo usas, y cuándo no

Úsalo con un databank en el que hayas corrido el cross-check SPP **con la opción *"Don't store data for 3D charts in Optimization profile"* desactivada**, y después de haber sacado los datos con `python3 -m sqx.export.export_spp`.

No sirve para comparar dentro y fuera de muestra. Y no es una limitación del programa: es que los datos no dan para eso. Dos tiradas SPP, una en IS y otra en OOS, **no se pueden emparejar**. Medido el 2026-09-19 sobre `Strategy 17.9.39`, con la misma estrategia y ajustes idénticos: de unas 11.600 combinaciones en cada tirada, **solo 6 coinciden**. El muestreador sortea, y sortea distinto cada vez. Ésa es exactamente la razón por la que existen las 5.000 variantes fabricadas a mano.

Antes de empezar

- **No toca SQX.** Lee archivos ya exportados, así que puedes lanzarlo con la GUI abierta y mientras haya un build corriendo.
- Tiene que existir el export: `~/Desktop/AlgoData/raw/<proyecto>/<databank>/<fecha>/spp/` con sus cinco CSV. Si no está, córrelo antes:

```
python3 -m sqx.export.export_spp --project XAUUSD --databank "SPP IS"
```

Cómo se ejecuta

```
python3 -m studies.breakage.spp.report --project XAUUSD --databank SPP_IS
```

flag	obligatorio	qué hace
`--project`	sí	nombre del proyecto tal y como aparece en SQX
`--databank`	no	carpeta del export, **con guiones bajos en vez de espacios**. Por defecto `SPP_IS`
`--day`	no	fecha del export a leer. Por defecto, el más reciente que tenga tabla SPP
`--strategy`	no	se puede repetir. Por defecto, todas las estrategias del export

Tarda unos segundos por estrategia.

Qué produce

Todo va a `~/Desktop/AlgoData/reports/<proyecto>/<databank>/<fecha>/spp/` (hasta el 25-09, suelto en la carpeta del día):

- **`estrategias/<nombre>.html`** / **`.json`** — el informe de cada estrategia (antes, una sección de `sppultra.md`), en el contrato de estudios que pinta la ventana.
- **`design_brief_<Estrategia>.json`** — uno por estrategia. Es el archivo que lee la etapa de fabricación de variantes. No lo edites a mano: se regenera cada vez que corres el comando.

Los informes **se acumulan** (una carpeta por fecha) y no se borra nada.

Cómo se lee el resultado

En pantalla te sale una línea por estrategia:

```
Strategy 1.19.29         proceed  n_eff= 4,283  max 17.01 vs nulo 10.04  vivos=10 congelados=0
Strategy 17.9.39         proceed  n_eff= 3,381  max 14.64 vs nulo  5.11  vivos=7  congelados=1
Strategy 23.16.37        proceed  n_eff= 4,300  max  8.44 vs nulo  5.35  vivos=10 congelados=1
Strategy 4.33.46         proceed  n_eff= 4,002  max 13.93 vs nulo  8.01  vivos=9  congelados=0
Strategy 41.5.25         proceed  n_eff= 4,320  max  7.58 vs nulo  4.85  vivos=12 congelados=0
```

- **`max`** **contra** **`nulo`** es el veredicto. `nulo` es lo que habría sacado la mejor de esas mismas combinaciones si ningún parámetro hiciera nada. `proceed` es que lo supera; `noise` es que no, y entonces **no hace falta seguir con esa estrategia**: te ahorras las 5.000 variantes.
- **`n_eff`** son las combinaciones con un resultado *distinto*. Siempre es menor que el número de filas, porque los parámetros muertos repiten el mismo backtest muchas veces. Es este número el que se usa para el umbral, no el de filas.
- **`congelados`** son los parámetros que el test de duplicados demostró que no hacen nada. Salen del diseño de variantes.

Dentro del informe, la tabla que más se mira:

parámetro	ReturnDDRatio	NetProfit	NumberOfTrades	grupos	idénticos	inerte
DICrossPeriod1	0.2929	0.4702	0.2698	224	0	
DICrossShift1	0.2359	0.0763	0.7847	189	0	
ExitAfterBars1	0.0264	0.0238	0.0691	184	0	
CBlock_SqzMmnInt21	0.0173	0.0193	0.0197	217	217	sí
IsBars1	0.0016	0.0007	0.0103	188	1	

Léela así, y en este orden:

- **`grupos`** **e** **`idénticos`** es la prueba. `grupos` son los pares de combinaciones que solo se diferencian en ese parámetro; `idénticos` son los que además dieron **exactamente el mismo resultado**. `CBlock_SqzMmnInt21`: 217 de 217. Ese parámetro no hace nada, está demostrado, y se congela.
- **Las columnas de números (η²)** son "qué porcentaje de la variación explica este parámetro". Y hay que leerlas con cuidado: fíjate en que `DICrossShift1` explica el **7,6 %** del beneficio neto pero el **78,5 %** del número de operaciones. **El mismo parámetro, dos respuestas muy distintas según qué métrica mires.** Por eso la tabla enseña varias y no una.

- **Por qué no se congela por η²**: mira `CBlock_SqzMmnInt21` otra vez. Está demostrado que no hace nada, y sin embargo su η² es 0,0173, más alto que el de `IsBars1` (0,0016), que sí hace cosas. Eso pasa porque el SPP sortea desequilibrado: a cada valor de un parámetro muerto le tocaron mezclas distintas de los demás, y esa diferencia se cuela en el η². **Fiarse del η² para congelar habría dejado dentro un parámetro muerto y fuera uno vivo.**

Y la tabla de mesetas:

```
| parámetro      | from | to   | width | center | argmax | original |
|----------------|------|------|-------|--------|--------|----------|
| DICrossPeriod1 | 40.0 | 46.0 | 3     | 43.0   | 43.0   | 67.0     |
| ExitAfterBars1 | 9.0  | 13.0 | 5     | 11.0   | 12.0   | 13.0     |
```

`center` es el **punto medio de la meseta**, no el mejor punto. Es lo que hace el `BestValue` del propio SQX. Cuando `width` es 1 no hay meseta: hay un pico, y el brief lo marca con `"spike": true`. Un parámetro que hay que acertar clavado no es un parámetro que puedas poner en real.

⚠ **Un pico puede ser falso.** Si la rejilla de pasos del SPP no pasa por el valor original de un parámetro (en las tres estrategias de USDJPY M30, `BBerDeviation1` va 1,89 · 2,01 · 2,13… y el original es 2,9), en ese nivel sólo está la propia estrategia original, y la tabla la da como argmax con `width` 1. Eso no es un pico: es la original sola. Está anotado en `OPEN.md` §79.

Las superficies por pareja (pestaña «Superficies por pareja»)

La tabla de mesetas mira los parámetros **de uno en uno**. Esta pestaña los mira **de dos en dos**: eliges en dos desplegables el parámetro del **eje X** y el del **eje Y**, entre todos los de la estrategia, y ves la rejilla de esa pareja. Están calculadas todas las parejas; si eliges el mismo parámetro en los dos ejes no hay rejilla.

Superficies por pareja

Elige un parámetro para cada eje. Meseta = el decil superior de las celdas de esa rejilla (10 %), la misma definición que la nube de parámetros. Una celda con pocas tuplas pesa lo mismo que una llena: una SPP muestrea desequilibrado. Si eliges el mismo parámetro en los dos ejes no hay rejilla.

Aquí se ve la combinación por defecto de Eje X (BBerPeriod1), Eje Y (ExitAfterBars1). Las demás están en el resultado JSON y en la ventana.

`x: BBerPeriod1 · y: ExitAfterBars1`

ExitAfterBars1 contra BBerPeriod1 — mediana de ReturnDDRatio

Cada celda es la mediana de ReturnDDRatio de todas las tuplas que usaron esa pareja de niveles, sean cuales sean los demás parámetros, θ_0 fuera. Cortes en los cuartiles y en el percentil 90: la banda de arriba es la meseta (≥ 3.82). Sin contar a θ_0, su celda (77 tuplas) vale 4.26, por encima del 94 % de las celdas, dentro de la meseta.

	32	34	36	38	40	42	44	46	48	50	52	54	56	58	60	62	64	66	68
11	3.46	1.08	1.31	0.98	1.58	1.55	1.43	1.58	2.10	3.50	2.58	2.48	1.10	0.55	0.995	1.38	1.06	1.31	0.5
12	1.83	1.58	0.155	0.52	1.22	2.79	1.69	1.51	2.10	3.58	2.33	1.95	2.34	0.14	0.34	0.45	1.40	0.88	0.93
13	1.35	1.26	0.72	0.7	1.14	1.75	1.29	1.55	0.77	3.42	2.35	1.56	2.08	0.965	0.02	0.43	1.08	0.685	0.92
14	2.16	1.27	0.84	0.84	1.65	1.37	3.04	2.48	3.41	3.51	2.76	3.59	2.37	0.93	0.93	1.13	1.78	1.36	1.81
15	2.12	2.19	1.48	1.22	2.21	2.31	3.06	2.38	4.15	4.37	4.03	5.09	3.79	1.70	1.20	1.25	1.61	1.66	0.91
16	1.88	1.60	1.46	1.64	2.19	1.75	2.92	2.57	4.21	5.12	5.13	6.33	4.58	1.75	2.00	2.14	2.62	1.64	1.86
17	1.70	1.38	1.25	1.33	1.72	2.31	3.30	2.20	3.84	3.85	4.08	3.81	2.62	2.45	2.05	2.68	2.33	2.34	1.83
18	1.68	1.02	1.08	1.01	1.75	2.90	2.98	2.61	4.51	4.26	5.03	6.21	4.33	2.19	2.58	2.80	2.21	2.55	2.56
19	1.27	1.11	0.705	0.95	1.81	1.94	2.40	2.57	3.98	5.13	4.31	3.03	4.16	1.64	1.30	1.89	1.61	1.54	1.61
20	1.02	0.93	0.87	0.77	1.26	1.64	1.50	2.99	2.35	5.13	4.11	7.60	2.67	1.20	1.40	1.05	1.27	1.00	1.15
21	0.7	0.45	0.07	0.53	1.15	1.07	1.02	1.09	1.77	3.53	3.69	4.42	2.04	0.79	1.17	0.8	1.74	1.27	1.21
22	1.05	0.63	0.225	0.535	1.22	0.97	1.39	1.46	2.88	3.32	3.73	3.99	3.67	0.59	1.34	1.46	1.90	1.42	0.95
23	0.775	0.69	0.17	0.52	1.00	0.7	0.9	0.825	2.35	3.58	3.05	4.81	3.15	1.73	1.18	1.01	1.03	1.77	1.47
24	0.905	0.56	0.695	0.45	0.925	0.38	1.07	1.41	2.29	3.02	2.92	3.67	2.56	1.35	1.14	1.32	0.85	1.30	1.19

< 1.07 1.61 2.57 3.82

`θ₀: fila 18, columna 50`

Estrategia 9.27.83 de `Test_USDJPY_donchianUpperCrossUp_M30`, SPP de IS, eje X `BBerPeriod1`, eje Y `ExitAfterBars1`.

- **Cada celda** es la mediana de `ReturnDDRatio` de todas las combinaciones que usaron esa pareja de valores, sean cuales sean los demás parámetros. **La estrategia original no cuenta** en ninguna celda: si contara, en su propia celda se votaría a sí misma.
- **θ_0** es la estrategia original, marcada en su celda (debajo de la rejilla: «θ_0: fila 18, columna 50»).
- **Los colores** cortan en los cuartiles y en el percentil 90 de las celdas de esa rejilla. **La banda más oscura es la meseta: el 10 % de arriba de las celdas** (tu definición, 2026-09-27; la misma que la nube de parámetros).
- **La nota de cada rejilla** te dice dónde cae θ_0: «Sin contar a θ_0, su celda (77 tuplas) vale 4,26, por encima del 94 % de las celdas, dentro de la meseta». Si pone «θ_0 está solo en su celda», el SPP no probó ninguna otra combinación con esos dos valores y la rejilla no dice nada de su vecindad.

Debajo, la tabla **«θ_0 en cada pareja»** lo resume: una fila por pareja, cuántas combinaciones hay en la celda de θ_0, el suelo de la meseta, el rango de θ_0 entre las celdas y si está en la meseta (`sí`, `no` o `sin vecinos`). En las tres estrategias de USDJPY, **todas las parejas con `BBerDeviation1` salen «sin vecinos»**: el SPP no volvió a probar nunca 2,9.

Cómo decidir con ella: si θ_0 cae fuera de la meseta en las parejas de los parámetros vivos, la estrategia está en un pico de dos dimensiones aunque cada parámetro suelto parezca estable. Es una razón para mirar con lupa el diseño de variantes, no para descartarla aquí: el veredicto de esta etapa sigue siendo el del ruido.

La pestaña va en cada `estrategias/<nombre>.json` / `.html` que escribe `report.py`, entre las mesetas y el diseño. La captura es de `python3 -m studies.breakage.spp.report --project Test_USDJPY_donchianUpperCrossUp_M30 --databank SPP_IS`, corrido el 2026-09-27.

Métricas para criterios (pestaña «Métricas para criterios») — sin umbral todavía

Desde el 2026-10-04 el estudio calcula además las métricas que pediste como criterios. **Ninguna tiene umbral y ninguna cambia el veredicto**: se ven en esta pestaña, son columnas del SPP en la tabla de Databanks y el autopiloto las lee como `spp.verdict_csv.<columna>` de `verdict.csv`.

- **Distancia del original a la mediana** (el criterio heredado de SQX), por separado en IS y en OOS1 y para las seis métricas del panel 1. Se publica de dos maneras: en **% de la mediana** (`distancia_pct_…`, el «± X %» de SQX; falta cuando la mediana está a menos de 0,25 IQR de cero, porque dividir por ella explota) y en **rangos intercuartílicos** de la rejilla (`distancia_IQR_…`). La segunda existe porque «± X %» premia la poca dispersión: una rejilla estrecha pasa aunque el original sea su extremo. Al lado se ve entre qué valores cae el 90 % central de las propias permutaciones: un original fuera de ahí es un extremo de su vecindad.
- **Decaimiento de la mediana** (`decaimiento_…`): mediana(OOS1) / mediana(IS) en Sharpe, Sortino, Profit Factor y R/Edge ratio. Es un cociente de medianas — IS y OOS1 no comparten permutaciones, no hay ratio por variante —; 1 es «no decae». Solo se publica si la mediana IS supera su suelo (0,1 en Sharpe y Sortino, 1 en Profit Factor y R/Edge ratio). Sharpe y Sortino son los anualizados de SQX (√252), comparables entre una IS de 8 años y una OOS1 de 4.
- **A de Vargha–Delaney** (`A_…`): probabilidad de que una permutación OOS1 al azar supere a una IS al azar. 0,5 = iguales; por debajo, decae. Es un tamaño de efecto, no un p-valor.
- **Wasserstein / IQR** (`Wasserstein_…`): cuántos IQR de la IS hay que mover una distribución para convertirla en la otra. Bilateral: también sube si la OOS1 es mejor.

Junto a cada valor la tabla enseña lo que leería **una familia sin ventaja**: la misma rejilla OOS1, con su forma, centrada donde cae una estrategia de Sharpe 0 con el ruido de un solo camino de precios (± 1/√años: ± 0,5 de Sharpe con 4 años). **Con 4 años de OOS1 ese rango cubre los valores reales de las diez estrategias del Nikkei** (A de Sharpe real 0,25 a 0,76; sin ventaja 0,00 a 0,75), así que hoy estas métricas no distinguen una estrategia buena de una sin ventaja: se quedan como diagnóstico y **no se ofrecen como criterio** en la zona Criterios hasta medirlas sobre un SPP de estrategias sin ventaja o con una OOS más larga. No hay p-valores: las permutaciones comparten un camino de precios y su número no es un tamaño de muestra. El p-valor del KS no se usa.

Un ejemplo completo

```
$ python3 -m studies.breakage.spp.report --project XAUUSD --databank SPP_IS --day 2026-09-10
Strategy 1.19.29         proceed  n_eff= 4,283  max 17.01 vs nulo 10.04  vivos=10 congelados=0
Strategy 17.9.39         proceed  n_eff= 3,381  max 14.64 vs nulo 5.11  vivos=7 congelados=1
Strategy 23.16.37        proceed  n_eff= 4,300  max 8.44 vs nulo 5.35  vivos=10 congelados=1
Strategy 4.33.46         proceed  n_eff= 4,002  max 13.93 vs nulo 8.01  vivos=9 congelados=0
Strategy 41.5.25         proceed  n_eff= 4,320  max 7.58 vs nulo 4.85  vivos=12 congelados=0

-> /home/sergioguslw/Desktop/AlgoData/reports/XAUUSD/SPP_IS/2026-09-21
```

Las cinco pasan el filtro de ruido. `Strategy 17.9.39` es la que lo pasa más holgada (14,64 contra 5,11, casi tres veces) y además es la única con un parámetro congelado además de `23.16.37`.

El brief que deja para esa estrategia dice, en resumen: siete parámetros vivos, uno congelado (`CBlock_SqzMmnInt21` fijo en 20), y los niveles a probar en cada eje — por ejemplo `DICrossPeriod1` en `[40, 43, 46, 49, 52, 58, 61, 64, 67]`, que es un abanico centrado en la meseta (43) pero **estirado hasta 67 para que quepa el valor original de la estrategia**. Eso es a propósito: si la meseta se ha movido, tiene que poder verse.

Qué NO te dice

- **No dice que la estrategia funcione.** Dice que su superficie de parámetros contiene más de lo que contendría buscando en ruido. Es un filtro para no gastar cómputo, no una aprobación.
- **No compara dentro con fuera de muestra**, por lo de las 6 combinaciones en común. Cualquier cosa que leas aquí es dentro de muestra.
- **El η² no es una medida limpia** en esta tirada, por el muestreo desequilibrado. Sirve para repartir niveles entre los parámetros vivos. No lo uses para decidir que algo no importa: para eso están las columnas `grupos` e `idénticos`.
- **La meseta es una estimación hecha con un solo tramo de historia.** Que un parámetro tenga una meseta ancha en 2008-2017 no significa que la tenga en 2018-2022. Precisamente eso es lo que va a medir la etapa siguiente.
- **`Ret/DD` crece con la longitud de la ventana.** Aquí da igual porque nunca se comparan dos ventanas, pero **no reutilices esta configuración para leer dos periodos distintos** sin cambiar la métrica.

Si algo falla

- **`IndexError` al buscar el export** — no hay ninguna carpeta con `spp/` dentro de `raw/<proyecto>/<databank>/`. Corre `sqx.export.export_spp` primero, o revisa que el databank lleve guiones bajos (`SPP_IS`, no `SPP IS`).
- **`KeyError` con el nombre de una estrategia** — el nombre tiene que ser exactamente el que usa SQX, espacios y puntos incluidos: `Strategy 17.9.39`.
- **Una estrategia con `congelados=0` y todos los `grupos` a 0** — el muestreador no llegó a generar pares que se diferencien en un solo parámetro. No es que no haya parámetros muertos: es que no hay pruebas. Hace falta más presupuesto de permutaciones.

18. Variantes — fabricar el lote de variantes de una madre

Qué pregunta responde

SQX no tiene ningún comando que cambie los parámetros de una estrategia ya hecha. Ni uno. Así que si quieres saber **qué pasaría si este indicador usara 61 periodos en vez de 67**, la única forma es escribir un `.sqx` nuevo con ese 61 dentro y volver a testearlo.

Este comando es la fábrica. Le das el **diseño** que salió del SPP (etapa 16) y te escribe en disco las variantes que ese diseño pide —entre 1.000 y 5.000, según cuántos parámetros se mueven—, más una tabla que dice qué combinación lleva cada archivo.

No decide nada sobre trading. Decide **qué combinaciones de entre cientos de miles** vale la pena probar, y luego las escribe. Desde el 2026-10-07 las reparte con un **hipercubo latino** dentro de la meseta que el SPP en muestra (`build`) midió: una muestra que cubre esa región por igual, sin esquinas ni montones en un punto, que es lo que necesitan el paso 17 (¿qué pasa si alguien elige dentro de esta familia?) y la nube de parámetros del 16.5.

Cuándo lo usas, y cuándo no

Úsalo cuando ya tienes el `design_brief_<Estrategia>.json` del SPP y el veredicto es `proceed`. Si el veredicto es `noise`, la familia entera no se distingue del azar y fabricar miles de variantes de ella es gastar medio giga y una tarde en medir ruido; el comando no te lo impide, pero te enseña el veredicto en la primera línea.

Fabricar el diseño entero es puro cálculo y escritura de archivos: no toca SQX. La única excepción, desde el 2026-09-29, es el **piloto** — un retest corto y automático de unas pocas docenas de segundos antes de fabricar el lote, que sí despierta el custodio (ver más abajo). `--no-pilot` o `--design-only` lo evitan si de verdad quieres el comando sin tocar nada.

No te da ningún resultado. Un archivo recién fabricado no se ha testeado: lleva dentro los números del padre hasta que SQX lo vuelva a correr. Si lees el `.sqx` fabricado esperando ver el resultado de la combinación nueva, vas a leer el del padre. Ver *Qué NO te dice*.

Antes de empezar

1. **Que exista el diseño.** Es el archivo que escribe la etapa 15:

```
python3 -m studies.breakage.spp.report --project XAUUSD --databank SPP_IS
```

Deja `~/Desktop/AlgoData/reports/XAUUSD/SPP_IS/<fecha>/design_brief_Strategy_17-9-39.json`.

2. **Que exista el export SPP del que salió ese diseño**, con sus CSV. La fábrica lo vuelve a leer, pero sólo cuatro columnas: de ahí saca los **centinelas**, que son combinaciones cuyo resultado SQX ya calculó.

3. **Sitio en disco.** Con el ajuste por defecto, 5.000 variantes (el tope del lote) ocupan **unos 505 MB**. Con `shape: minimal` bajan a 70 MB y con `full` suben a 631 MB.

4. No hace falta cerrar SQX ni parar nada a mano: el piloto arranca y para el custodio solo, igual que la etapa siguiente. Con `--no-pilot` o `--design-only` el comando no habla con SQX en absoluto.

Cómo se ejecuta

```
python3 -m sqx.variants.make \
  --brief ~/Desktop/AlgoData/reports/XAUUSD/SPP_IS/2026-09-21/design_brief_Strategy_17-9-39.json \
  --project XAUUSD
```

flag	obligatorio	qué hace
`--brief`	sí	ruta del `design_brief_<Estrategia>.json`. De él salen los parámetros vivos, el tramo de meseta de cada uno (`plateau: {from, to}`, sólo del SPP en muestra: un brief del SPP fuera de muestra se rechaza), su clase (`live`, `frozen` o `shift`) y los congelados
`--project`	sí	nombre del proyecto, sólo para decidir la carpeta de salida
`--limit N`	no	fabrica sólo las N primeras filas. **El diseño no cambia**: se calcula el lote entero y se escriben N. Es lo que se usa para probar la cadena con tres archivos antes de comprometerse a un lote entero
`--design-only`	no	calcula el diseño, escribe `plan.csv` y para. No escribe ningún `.sqx`
`--no-pilot`	no	salta el piloto (ver abajo) aunque `config.yaml` lo tenga encendido

Tarda **unos 50 segundos** en escribir 5.000 con el ajuste por defecto (6 segundos con `minimal`). El diseño en sí es instantáneo.

Los mandos que no son flags están en `sqx/variants/config.yaml`: la semilla, cómo se reconstruye el rango de un parámetro congelado, cuántos centinelas, y **la forma del archivo** (`minimal` / `no_profile` / `full`). **El tamaño del lote** son tres números del ledger (`variants.n_min` 1.000, `variants.n_max` 5.000, `variants.per_coefficient` 8), editables desde la ventana: el lote tiene `min(n_max, max(n_min, 8 × p))` variantes, con `p = 1 + 2k + k(k−1)/2` los coeficientes de una superficie cuadrática sobre los `k` parámetros vivos. Si se llega al tope, el informe avisa de que hay menos de 8 variantes por coeficiente.

El piloto: unas pocas docenas de segundos antes de fabricar el lote entero

Medido el 2026-09-24 en `Strategy 17.9.39`: de 2.000 variantes fabricadas, **1.002 operaron menos de 30 veces dentro de muestra**. No es una cola, es la mitad del diseño — presupuesto gastado en combinaciones que ni siquiera son un punto de una superficie de rendimiento, y que además hay que descartar después, dejando parámetros con un solo valor útil.

Por eso, **desde el 2026-09-29 y por defecto**, antes de fabricar el lote el comando fabrica y retestea unos cientos de tuplas de muestra (las que se mueven, nunca las congeladas) en el mismo proyecto — el mismo retest corto que usa la etapa siguiente, no uno nuevo — y cuenta cuántas veces operó cada una. Si un valor de un parámetro deja a sus tuplas por debajo de 30 operaciones de mediana (con suficientes puntos para fiarse), ese valor se descarta **antes** de fabricar el lote grande — nunca el parámetro entero, y nunca el valor con el que se construyó la madre.

`--no-pilot` lo salta (por ejemplo para una prueba de humo con `--limit`); `pilot.enabled: false` en `sqx/variants/config.yaml` lo apaga para siempre. **Nunca es silencioso**: `pilot.json` queda junto al lote con la mediana de operaciones y el soporte de cada nivel, y `design.json` lleva el mismo recorte bajo `pilot_dropped` — dos lotes de la misma madre, uno con el piloto y otro sin él, no son el mismo diseño y `pilot_dropped` es lo primero que hay que mirar para saber en qué se diferencian.

```
piloto: 287 reteseadas de 300, 2 niveles descartados en 1 parametros (< 30 operaciones en build)
```

El piloto también se puede correr suelto, para mirarlo sin fabricar nada más (`python3 -m sqx.variants.pilot --brief <design_brief.json> --project <PROYECTO> --out <carpeta>`) — es lo que `make` llama por dentro.

Qué produce

Todo en `~/Desktop/AlgoData/strategyPermutations/<proyecto>/<Estrategia>/` :

archivo	qué es
`pilot.json`	el piloto: cuántas tuplas de muestra, cuántas volvieron, qué niveles se descartaron y por qué. Ausente si se saltó
`plan.csv`	el diseño completo: una fila por combinación, con su estrato (`lhs` , `origin` , `canary` o `inert`) y su identificador, aunque fabriques sólo 3
`sqx/P00000.sqx` …	las estrategias. Una por fila fabricada
`manifest.parquet`	**la tabla que une todo el estudio.** Una fila por archivo que hay de verdad en `sqx/` , leída del archivo, no del plan

Se sobrescribe. Hay un diseño vigente por estrategia y volver a correr el comando lo reemplaza, para que no pueda existir la pregunta "¿cuál de estas cinco mil carpetas es la buena?". Si borras `sqx/` a mano y vuelves a lanzar, el manifiesto se reconstruye con lo que haya.

Cómo se lee el resultado

Lo primero que sale son los costes vigentes del símbolo — es obligatorio y sale siempre:

```
# XAUUSD — overrides to apply

SQX symbol: XAUUSD_DukasM1_Infinox   verified: 2026-09-03

- **spread**: use `10.0` points (SQX default `10.0`) — PROVISIONAL 2026-09-21 — SQX default, not an
agreed Infinox figure. Owner will replace it.
- **commission**: use `8.0` $ per lot per side (SQX default `SizeBased 8`) — PROVISIONAL 2026-09-21 —
...
```

Luego el diseño:

```
Strategy 17.9.39  verdict=proceed  live space 3,888 tuples, with the frozen 19,440
  lhs            asked 1,000  kept 1,000  (6 live parameters, 3,888 tuples in the plateau)
  controls                    kept     5
```

Se lee así:

- **`live space 3,888`** — con los niveles que dejó la meseta, **todas** las combinaciones posibles de los parámetros vivos son 3.888. `with the frozen 19,440` es lo mismo contando también los parámetros congelados.
- **`lhs asked / kept`** — lo que pidió la fórmula del tamaño y lo que quedó tras redondear los enteros: **nunca se repite una combinación**. Si el hipercubo no da tantas combinaciones distintas (la meseta es pequeña), `kept` es menor y sale `⚠ N fewer than asked` ; no es un error, es información: `design.json` lo guarda como `shortfall` . Si el tope de 5.000 corta el lote, sale `⚠ the cap … cut the batch` (`capped`).
- **`controls kept 5`** — los controles, que no son material de análisis (ver más abajo).

Y por último la fabricación y la verificación:

```
wrote 3 .sqx as 'no_profile', 0.3 MB
{
  "planned": 3,
  "on_disk": 3,
  "missing": [],
  "unexpected": [],
  "tuple_mismatch": [],
  "duplicate_tuples": 0
}
```

Esos cinco números son lo importante de todo el comando. No los calcula mirando lo que quería escribir: abre los archivos de vuelta, saca de dentro los parámetros que llevan y los compara.

campo	qué significa si no es cero / vacío
`missing`	el plan tenía un identificador que no llegó a ningún archivo
`unexpected`	hay un archivo en la carpeta que el plan no pidió — típicamente restos de una tirada anterior
`tuple_mismatch`	**el peor de todos.** El archivo `P01234` no lleva dentro la combinación que el plan dice que lleva. El estudio entero se une por ese identificador
`duplicate_tuples`	dos archivos con la misma combinación. SQX los puede fundir en uno y quedarte con filas del manifiesto sin archivo detrás

Si alguno falla, el comando **sale con error** y te dice que no cargues el lote. Todo vacío y a cero es lo que tiene que salir siempre.

Un ejemplo completo

Probar la cadena con tres archivos antes de comprometerse a un lote entero:

```
$ python3 -m sqx.variants.make \
    --brief ~/Desktop/AlgoData/reports/XAUUSD/SPP_IS/2026-09-21/design_brief_Strategy_17-9-39.json \
    --project XAUUSD --limit 3

[... los costes del símbolo ...]

Strategy 17.9.39  verdict=proceed  live space 3,888 tuples, with the frozen 19,440
  lhs            asked 1,000  kept 1,000  (6 live parameters, 3,888 tuples in the plateau)
  controls                    kept     5

wrote 3 .sqx as 'no_profile', 0.3 MB
{
  "planned": 3,
  "on_disk": 3,
  "missing": [],
  "unexpected": [],
  "tuple_mismatch": [],
  "duplicate_tuples": 0
}

-> /home/sergioguslw/Desktop/AlgoData/strategyPermutations/XAUUSD/Strategy_17.9.39
```

Y mirando el manifiesto, que es lo que leerán todos los análisis de después:

```
variant_id  sqx_name                 stratum  origin  param_DICrossPeriod1  ...
canary_expect_netprofit  canary_expect_trades
P00000      Strategy 17.9.39 P00000   origin   True                    67.0
NaN                 <NA>
P00001      Strategy 17.9.39 P00001   canary   False                   43.0
38896.359375                 1077
P00002      Strategy 17.9.39 P00002   canary   False                   94.0
-37128.390625                 843
```

Las tres primeras filas de cualquier lote son **controles**, no material de análisis:

- **`P00000` es la madre**, la estrategia original reescrita por la fábrica sin cambiarle nada. Si la reescritura rompiese algo, es la fila que deja de parecerse a la estrategia de la que salió. Y es la única que la etapa de borrado **se niega a tocar**.
- **`P00001` y `P00002` son centinelas**: combinaciones que SQX ya probó en el SPP, con el resultado que dio guardado al lado. Están elegidas en los extremos de la rejilla — la de más beneficio y la de más pérdida — a propósito: una cadena mal montada devuelve un número intermedio creíble, no devuelve el máximo de una rejilla de cuatro mil.
- Y después hay **una pareja por cada parámetro congelado**: la madre con ese parámetro movido a tope. Como el diseño lo congeló por haber demostrado que no mueve nada, esas filas tienen que volver **exactamente iguales a `P00000`**. Es la única comprobación de que congelarlo estaba bien.

Cuando la etapa siguiente recoja los resultados, si los centinelas no vuelven con lo esperado, la cadena está rota en algún punto — y te enteras antes de mirar cinco mil resultados.

Las operaciones de todas las variantes: `variant_trades.parquet`

Desde el 2026-10-07 el paso 16.5 guarda además **las operaciones de cada variante**, en las tres patas (`build` , `oos1` , `oos2`) y en todos los mercados: el paso 17 y la nube de parámetros miden el **exceso** de cada variante sobre la deriva del mercado (el mismo del paso 14) y para eso necesitan sus operaciones, no sólo su curva. Esto **revierte** la decisión de 2026-10-04 de guardar sólo las de la madre. Cuesta unos **90 minutos por lote** (medido en la corrida de referencia); el piloto automático lo lanza solo, tras recoger el lote y antes de vaciar los databanks:

```
python3 -m sqx.variants.varianttrades --work <carpeta del lote> --project <PROYECTO>
```

Reúne los tres exports de `export_retest --batch variants` en `variant_trades.parquet` , en la carpeta del lote: las columnas de una operación más `variant_id` , `leg` (`build` / `oos1` / `oos2`) y `market` (`Main` para el mercado principal, el feed para los demás), con los controles incluidos y marcados en el manifiesto. Después **borra** los exports por pata: ese fichero es su único guardián. `--day` fija la fecha del export (hoy, si no se pone). No usa SQX y es rápido; lo caro es el export previo.

Todavía no existe ningún `variant_trades.parquet` de un lote real: el comando está construido y probado con lotes sintéticos, y su salida real se verá en la primera corrida (OPEN.md, parte V del encargo 48). La captura es la ayuda del comando.

```
$ python3 -m sqx.variants.varianttrades --help
usage: varianttrades.py [-h] --work WORK --project PROJECT [--day DAY]

Every variant's trades on the three legs and every market, in one
`variant_trades.parquet` per batch.

options:
  -h, --help         show this help message and exit
  --work WORK
  --project PROJECT
  --day DAY          export date; today when absent
```

Los otros comandos de la carpeta

Los usa el piloto automático; sólo los llamas a mano para probar:

comando	para qué
`python3 -m sqx.variants.harness --kind spp_is --project <PROYECTO> --output <banco> …`	reescribe una tarea ya configurada del proyecto de la madre (SPP en muestra, SPP fuera de muestra o un retest con mercados adicionales) a partir de una tarea de referencia que ya corrió; la tarea se busca por su **título**, nunca por el nombre del fichero
`python3 -m sqx.variants.spp --work <carpeta> --mother <madre.sqx> --kind spp_is --chart '…'`	corre el SPP de reconocimiento de una madre en el custodio y deja el perfil donde el exportador lo encuentra
`python3 -m sqx.variants.mothertrades --work <carpeta> [--work …]`	las operaciones **de la madre** (`P00000`) en cada pata, en un `trades.parquet` por lote: de ahí reconstruye el paso 20 su curva de `oos2` . Es distinto de `variant_trades.parquet` , que lleva las de todas las variantes

Qué NO te dice

- **No dice si una variante es buena.** No hay ni un número de rendimiento aquí. Esto escribe archivos; medirlos es la etapa siguiente.
- **Un `.sqx` fabricado lleva dentro los resultados del padre** hasta que SQX lo vuelva a testear: las operaciones, la curva de equity y las estadísticas guardadas son las de la madre. Con la forma `no_profile` (la de por defecto) y con `full` eso es así. Consecuencia incómoda: **una variante que por lo que sea no llegue a correr no se ve vacía, se ve como la madre.** El manifiesto no puede detectarlo; los centinelas y el conteo del databank sí.
- **Las expectativas de los centinelas son sobre la muestra IS**, y sólo valen mientras la ventana dentro de muestra del retest sea la misma sobre la que corrió el SPP. Un retest configurado con otra historia falla todos los centinelas por un motivo que no tiene nada que ver con la fábrica.
- **Los niveles de un parámetro congelado son una reconstrucción, no un dato.** El diseño congela un parámetro y da su valor, no un rango; las parejas inertes necesitan variarlo y el rango se rehace igual que lo hace SQX en sus propias permutaciones (±30 %; un *shift* no se mueve nunca). Es la suposición más discutible del módulo y está anotada como tal.
- **El lote cubre la meseta del SPP en muestra y nada más** (dueño, 2026-10-07). Un parámetro entero toma enteros dentro de `[from, to]` , uno decimal hasta 3 decimales; los *shift* y los congelados se quedan en el valor de la madre en **todas** las variantes. Ya no hay estratos de vecindad, factorial y cobertura, ni «al menos 1.000

distintas por ensanchamiento»: si la meseta no da 1.000 combinaciones distintas, el lote es más pequeño y lo dice.
- **Que el diseño contenga la combinación buena no está garantizado.** Una meseta que se haya movido fuera de muestra más allá del tramo medido en `build` se queda fuera.

Si algo falla

`not variables of this strategy: ['X']` — el diseño nombra un parámetro que la estrategia madre no tiene. Casi siempre es un `--brief` de una estrategia y un `.sqx` de otra: el `.sqx` se coge del campo `source` del propio diseño, así que si has movido o editado carpetas del export a mano, se descolocan. Vuelve a correr la etapa 15.

`settings.xml does not carry both name fields` — el `.sqx` madre no tiene los dos campos de nombre donde se esperan. Es señal de que la versión de SQX que lo escribió no es la de esta caja; no fuerces nada, dilo.

`the batch on disk is not the batch that was planned` — la verificación encontró algo. No cargues ese lote en SQX. Mira cuál de los cuatro campos salió con contenido; lo normal es `unexpected`, que son restos de una tirada anterior en `sqx/`.

Un `FileNotFoundError` sobre `spp.parquet` — falta el export SPP del que salió el diseño. Los centinelas salen de ahí. Reexpórtalo con `python3 -m sqx.export.export_spp`.

37. El retest de las variantes para el WFC y el CSCV

Escribe en un proyecto custom de SQX **las tres tareas** que producen el material de los pasos 17 (Walk Forward Correlation) y 18 (CSCV): una por tramo — `build`, `oos1`, `oos2` —, cada una con el spread y el slippage que le tocan, y cada una corriendo además los mercados adicionales.

Qué pregunta responde

Ninguna, todavía. Esto no analiza: **prepara los backtests**. Lo que produce son tres databanks con las mismas variantes corridas sobre tres tramos distintos, que es el material que luego leen `studies/optimisation/wfc/` (¿el in-sample predice el out-of-sample?) y su CSCV (¿mi forma de elegir parámetros sobreajusta?).

Los dos estudios comparten configuración a propósito: leen el mismo panel y la misma equity diaria del mismo lote. Si cada uno tuviera su bloque en `assets/_build.yaml` serían dos lotes distintos y sus respuestas ya no hablarían del mismo objeto. Por eso hay **un solo bloque** `wfc:`.

Cuándo lo usas, y cuándo no

Después de fabricar el lote de variantes (`/variants`, paso 16.5) y antes de correrlo. Nunca antes: las variantes son los ficheros que se van a retestear.

⚠ **Gasta `oos2`.** `assets/_policy.yaml` reserva ese tramo al WFC y a la WFM, y cada mirada lo gasta. Esto es el WFC, así que es uno de sus dos consumidores legítimos — pero no se corre "para ver qué sale", y el paso 19 sigue sin poder leerse hasta que 17, 18 y 19 estén los tres hechos.

Antes de empezar

1. El proyecto tiene que ser **custom**, clonado con `sqx/projects/builder.py`, nunca el `Retester` de serie (regla dura 10).
2. El proyecto tiene que estar **cerrado**: SQX reescribe el `.cfx` al salir y se pierde el cambio en silencio (regla dura 4). El comando lo comprueba y se niega.
3. `python3 -m core.assets XAUUSD` en verde (regla dura 5).

Cómo se ejecuta

```
python3 -m sqx.projects.wfc XAUUSD \
    --cfx ~/Desktop/SQX_w2/user/projects/XAUUSD_WFC/project.cfx \
    --timeframe M30 \
    --tasks Retest-Task1.xml,Retest-Task2.xml,Retest-Task5.xml
```

flag	obligatorio	qué hace
`--cfx`	sí	el proyecto custom donde se escriben las tres tareas
`--timeframe`	sí	el timeframe del proyecto; es el que llevan los mercados adicionales
`--tasks`	sí	tres ficheros de tarea **en el orden de** `wfc.tasks`: build, oos1, oos2. Se les cambia el título al de la doctrina y se activan
`--no-markets`	no	deja fuera los mercados adicionales en las tres patas de **este** proyecto, diga lo que diga `wfc.markets`. Para un estudio que sólo lee el mercado principal (el paso 24, `54-atr-calculator.md`)
`--json`	no	la misma salida en JSON

Salida real sobre una copia del donante congelado:

```
XAUUSD  XAUUSD  <- WFC Variants (las tres leen el MISMO lote)
  ✓ WFC 1 IS     build 2008.01.01→2017.12.31  -> WFC Build  1 condicion(es) apagada(s)
      mercados: XAGUSD_DukasM1_Infinox, BRENTCMDUSD_ftmo
  ✓ WFC 2 OOS1   oos1  2018.01.01→2022.12.31  -> WFC OOS1   1 condicion(es) apagada(s)
      mercados: XAGUSD_DukasM1_Infinox, BRENTCMDUSD_ftmo
  ✓ WFC 3 OOS2   oos2  2023.01.01→2026.08.30  -> WFC OOS2   1 condicion(es) apagada(s)
      mercados: XAGUSD_DukasM1_Infinox, BRENTCMDUSD_ftmo
```

Por qué tres tareas y no una

Una sola tarea de 2008 a 2026 llevaría **un** spread y **un** slippage para los dieciocho años, y habría que elegir cuál miente menos. Con tres, cada tramo cobra lo suyo: comprobado en el `.cfx` escrito, la tarea de `build` sale con spread 11.0 en la plata y slippage 2.5, y la de `oos2` con 22.0 y 5.

Y además sale gratis en información. Las tres leen el **mismo** databank de entrada (`WFC Variants`) y cada una escribe el suyo (`WFC Build`, `WFC OOS1`, `WFC OOS2`), así que el corte in-sample / out-of-sample deja de ser una fecha dentro de una curva y pasa a ser **de qué tarea vino** cada curva. El `<OutOfSample>` de cada tarea se vacía justamente por eso: una tarea que corrió un tramo entero *es* una muestra.

Los mercados adicionales van dentro

`markets: true` en la doctrina. Cada una de las tres tareas lleva también el crosscheck `RetestOnAdditionalMarkets` con los mercados de `assets/_markets.yaml`, cada uno con **sus** costes y recortado a su `data_from` si empieza más tarde que el tramo — en la tarea de `build`, el Brent corre 2013→2017 y la plata 2008→2017.

Son los **crossmarkets**, no los crossTF: un timeframe reescalado es la misma serie de precios remuestreada, así que no añade grados de libertad a un CSCV.

⚠ **El coste se multiplica por (1 + nº de mercados).** Con XAUUSD y sus dos mercados family, las tres tareas son **nueve** backtests por variante. Mira el tamaño del lote antes de arrancar.

Aquí tampoco se filtra nada

`wfc.conditions: []`. El comando apaga todas las condiciones de aceptación del crosscheck, fuerza `DeleteFailedStrategies` a false y pone `retestSelected="false"`, y dice cuántas apagó. Una variante descartada y una variante que perdió dinero son la misma cosa para el que cuenta, y entonces el PBO no significa nada.

Qué hay que hacer a mano todavía

Los tres databanks tienen que **existir** en el proyecto (`WFC Variants`, `WFC Build`, `WFC OOS1`, `WFC OOS2`): SQX empareja por el nombre exacto y **ignora en silencio** un databank que ningún `<Databank>` del `config.xml` declara. Créalos en la GUI del proyecto custom antes de correr nada, o clona un donante que ya los lleve.

Qué NO hace

No carga las variantes, no arranca el custodio y no cosecha nada. Eso es `/variants` y `sqx/variants/execute.py`. Esto sólo deja el proyecto escrito.

19. Walk forward correlation (paso 17) — ¿elegir los parámetros por el pasado sirve fuera?

Qué pregunta responde

Si ordenas las versiones de una estrategia por lo bien que fueron dentro de muestra, ¿lo que elegirías sigue ganando fuera?

El estudio coge una estrategia madre, fabrica muchas versiones con parámetros distintos (las *variantes*, capítulo 18) y las mide todas en dos tramos de la historia: uno que un optimizador ya habría visto (**dentro**) y otro que no (**fuera**). Mira dos cosas:

- **El orden.** ¿Las que ganan dentro tienden a ser las que ganan fuera? Se mide con la correlación de rangos de Spearman (ρ): +1 es el mismo orden dentro y fuera, 0 ningún parecido, −1 el orden invertido. Una ρ sola no dice nada: hay que compararla con lo que daría el puro azar (ver abajo).
- **El nivel.** El 20 % de variantes que mejor fueron dentro es el «top», lo que elegiría un optimizador. ¿Qué gana ese top fuera, en mediana? Si pierde, ningún orden bonito te salva. Y si gana, el nivel **gana**. No se pide significación (dueño, noche del 2026-10-07: la tabla de la propuesta tal como está escrita). El **p del nivel** (fracción de sorteos del nulo cuya mediana del top iguala o supera la observada) se anota al lado y no decide.

Todo se mide en **exceso por año**: el beneficio neto de costes de cada operación menos lo que el mercado pagó por estar dentro durante esa operación (la deriva, paso 14), sumado por variante y dividido por los años de la ventana. Hay dos lecturas: **bruta** (antes de comisión, swap, spread y deslizamiento), que sirve para el **orden**, y **neta**, que sirve para el **nivel**. Los costes solos ya ordenan parecido dentro y fuera (quien opera más paga más), y el nulo no lo ve; por eso el orden se mide antes de costes.

El veredicto: PASA, LIMBO, NO PASA

orden \ nivel	la mediana del top fuera es > 0	la mediana del top fuera es ≤ 0
positivo o neutro	**PASA**	**NO PASA**
perverso	**LIMBO**	**NO PASA**
sin dato (o menos de 30 variantes)	**LIMBO**	**LIMBO**

- **Positivo**: el orden de dentro se repite fuera más de lo que daría el ruido. **Perverso**: lo mejor de dentro tiende a ser lo peor de fuera (optimizar empeora). **Neutro**: ni una cosa ni otra. «Fuerte» es un orden positivo con ρ ≥ 0,30: una etiqueta, no cambia el estado.
- **PASA no es un voto a favor, es un veto levantado** (dueño, 2026-10-07): dice que, dentro de esta familia, elegir por el pasado no lleva a algo que pierda fuera. No dice que la madre tenga ventaja.
- **El paso no corta nada.** Escribe un veredicto y una fila en el registro; no borra ninguna estrategia. El paso 20 lo traduce así: PASA = sin fallo, NO PASA = fallo, LIMBO = indeciso.

Cuándo lo usas, y cuándo no

Lo usas cuando ya hay un lote de variantes de una madre retesteado en tres tramos (`build`, `oos1`, `oos2`) y con las operaciones de cada variante exportadas. Es el paso 17 del workflow.

No lo usas para elegir parámetros: no te dice cuáles poner, te dice si elegir por el pasado sirve en esta familia. La otra pregunta, «¿mi manera de elegir es propensa al sobreajuste?», es del CSCV (paso 18, capítulo 25).

No lo usas con pocas variantes: con menos de 30 tras los filtros el resultado es «sin dato» y el estado LIMBO. Significa «fabrica más», no «baja el umbral».

Antes de empezar

1. **Un lote con operaciones.** La carpeta del lote tiene que contener `metrics.parquet` **y** `variant_trades.parquet` (las operaciones de cada variante en cada tramo). Cómo se produce: el diseño y las variantes en el capítulo 18 (paso 16.5), el retest de las tres tareas en el 37 y su cosecha con `/variants`. **Hoy no existe ningún lote real con `variant_trades.parquet`**: el estudio está construido y probado con familias fabricadas, pero nunca se ha corrido de punta a punta con datos reales (OPEN.md, parte V).
2. **`oos2` está sellado.** El veredicto se juzga sobre `oos2`, el único tramo que nadie ha mirado. El sello (`ledger.gate.seal`) lo abre solo si existen a la vez una regla para el paso 17 en `criteria.yaml` **y** un registro de calibración con `abre: true` firmado para esa misma regla. El registro de calibración (`AlgoData/ledger/wfc_calibration.json`) dice hoy `abre: true` (2026-10-07), pero **midió la alternativa** (el nivel contra su propio nulo), no el criterio escrito: léelo con la advertencia de `calibrate/README.md` y del capítulo 107. Si falta la regla o el registro dice `abre: false`, el comando se para en el sello y no lee nada de `oos2`.
3. Los costes del símbolo acordados (`python3 -m core.assets <SÍMBOLO>` sale con código 0).

Cómo se ejecuta

```
python3 -m studies.optimisation.wfc.report --work ~/Desktop/AlgoData/<carpeta-del-lote> --family
<FAMILIA>
```

Sin captura: la salida nueva (tres composiciones, estado por orden y nivel) solo se ha producido con datos sintéticos; ningún lote del directorio de datos tiene `variant_trades.parquet`. Las antiguas `--inside`, `--outside` y `--split` ya no existen.

flag	obligatorio	qué hace
`--work`	sí	la carpeta del lote: `metrics.parquet` y `variant_trades.parquet`
`--family`	sí	la familia de plantillas del lote (tercera parte del identificador del estudio en el registro). Activo y marco temporal se leen del propio lote
`--set sección.clave=valor`	no	cambia un parámetro de `config.yaml` solo para esta corrida

Es puro cálculo sobre ficheros: **no toca SQX**, se puede lanzar con cualquiera de las instalaciones abiertas. Cada lectura apunta una fila por tramo en el registro global (paso 17), así que mirar gasta: no se repite «para ver».

Las tres composiciones

El estudio mide tres maneras de partir la historia, cada una con su propio nulo:

composición	dentro	fuera	papel
c1	build + oos1	oos2	**decide**
c2	build	oos1 + oos2	se anota al lado
c3	build	oos2	se anota al lado

Las tres leen `oos2`, así que **que coincidan no es una confirmación**: es la misma ventana de fuera mirada con distinto dentro. Solo el estado de c1 llega al piloto.

Los filtros: qué variantes entran

Antes de medir se quitan, en este orden, y el informe cuenta cuántas cae en cada filtro:

1. los **controles** (variantes canario y pares inertes que el diseño añade para vigilar el método);
2. las que operan poco **dentro**: suelo de 15 operaciones por cada parámetro vivo (k);
3. los **duplicados exactos** (la misma lista de operaciones, no el mismo total: dos variantes distintas que sumen igual al céntimo se quedan);
4. las que operan poco **fuera**: suelo de 10 operaciones por año de ventana;
5. las que no tienen operaciones exportadas.

Un suelo que la propia madre no cumple no se aplica, y se avisa arriba. Si tras los filtros quedan menos de 30 variantes: «sin dato».

Qué produce

Una carpeta `estudios/` dentro del lote con **un solo informe para las tres composiciones**:

- `estudios/wfc.json`: todas las cifras (`facts.c1`, `facts.c2`, `facts.c3`; el `estado` solo en c1).
- `estudios/wfc.html`: lo que se lee, con la nube, las tablas y la ayuda de cada columna.
- `estudios/wfc.md`: lo mismo en texto.

Y las filas del registro (una por tramo). Sobrescribe el informe anterior del mismo lote.

Cómo se lee el resultado

1. **Arriba, el estado de c1** y la frase de su casilla de la tabla. Es lo que cuenta.
2. **La nube**: un punto por variante, exceso por año dentro (horizontal) contra fuera (vertical). Los cuadrantes dicen lo que la ρ no ve: una nube inclinada puede estar entera en el cuadrante que gana dentro y pierde fuera.
3. **La tabla de cifras**: ρ y su rango en el nulo (p5–p95); `p+` y `p−` son las fracciones de sorteos del nulo con una ρ igual o mayor / igual o menor que la observada. Con corte 0,05: `p+ ≤ 0,05` es orden positivo, `p− ≤ 0,05` perverso, el resto neutro.
4. **El nivel del top** (mediana fuera del top de dentro), su **p del nivel** y el nivel de todas las variantes. Por encima de cero gana el top; con p ≤ 0,05 gana más que por azar.
5. **R_sel**: qué parte de la ventaja del top dentro se conserva fuera (1 = entera, 0 = elegir no sirve, negativo = selección perversa). Se escribe «sin dato» cuando la ventaja de dentro es casi nula (menos del 5 % del rango intercuartílico). Es provisional y no decide.
6. **El recuento por filtro**: cuántas variantes cayó cada uno. Si cae casi todo el lote, la ρ habla de una porción rara del espacio de parámetros.

El nulo. Se sortea un signo para cada semana de fuera, **el mismo para todas las variantes**: la familia conserva su parecido interno y pierde su ventaja, y se repite 2.000 veces. Es la ρ que verías si fuera no hubiera nada. Cada operación cuenta entera en la semana en que cierra. Si más del 10 % de las operaciones de fuera cierran un lunes o martes tras un fin de semana abierto, el informe avisa y repite el nulo con bloques de 2 semanas (`p_mas_bloque`, `p_menos_bloque`, `orden_bloque`), solo anotado.

Anotaciones (nada decide): el p del nivel, el nulo con bloques de 2 semanas, la ρ sin los suelos de operaciones (`rho_sin_suelos`), la ρ parcial quitando lo que explica el número de operaciones, el nivel con la deriva tomada solo de barras anteriores a `oos2`, los cuatro cuadrantes (++, +−, −+, −−, en neto), el recuento por filtro y `descuadre_sqx` (que el neto recalculado reproduce el `NetProfit` de SQX).

Un ejemplo completo

Con un lote real sería: sello abierto → `report` → leer el estado de c1 → ver la nube. Ningún lote real ha pasado todavía por el paso 17: el primero que lo haga será la primera lectura de `oos2`. La calibración (capítulo 107) usa familias fabricadas; la del 2026-10-07 midió la alternativa con nulo en el nivel, no esta tabla.

Las dos trampas medidas

- **«La mediana del top es mayor que cero» aprueba la mitad de las familias sin ventaja, y ese es el criterio.** La mediana de un ruido simétrico es positiva la mitad de las veces: una familia sin ventaja es PASA el 0,5 × (1 − α) de las veces por construcción (medido 0,50 en 300 familias). El dueño lo sabe y mantiene la tabla: el PASA del paso 17 solo levanta un veto.
- **Una ventaja plana con operaciones desiguales finge un orden después de costes** (y, si la ventaja es por operación, también antes). Con familias sin orden real: con el mismo nivel bruto por año para todas, el orden sale «positivo» el 3 % de las veces en bruto y el 23 % en neto; con la misma ventaja bruta por operación, el 75 % en bruto y el 30 % en neto. Bruto quita el artefacto de costes, no el de «opera más, gana más», que anota la ρ parcial.
- **La dependencia entre variantes mueve el LIMBO y el NO PASA con ventaja, no el PASA sin ella.** Variantes que son copias unas de otras ensanchan el nulo, así que una ventaja real se ve peor; la tasa de falsos PASA se queda donde la deja la construcción.

Preguntas abiertas del dueño

> **R_sel** (la parte de la ventaja del top que se conserva fuera) es provisional: el dueño aún no ha decidido su umbral de «sin dato». Y quedan en `POSSIBLE_IMPROVEMENTS.md` de la carpeta del estudio las elecciones de modelado que tenían alternativa real; léelo antes de cambiar una.

Qué NO te dice

- **No dice que la estrategia vaya a funcionar hacia delante.** PASA es que el orden de dentro y el top no fallaron fuera, en la ventana leída.
- **No dice que la madre tenga ventaja**: PASA levanta un veto, no vota a favor.
- **No juzga la manera de elegir en toda la historia** (eso es el CSCV, capítulo 25) ni la estrategia en sí (eso es la WFM).
- **No hay confirmación entre composiciones**: las tres leen `oos2`.
- **Un solo resultado no es una tasa de error**: cuánto se equivoca la tabla lo dice la calibración, no esta corrida.

Si algo falla

- **El comando se para en el sello de `oos2`.** Falta la regla del paso 17 en `criteria.yaml` o el registro de calibración dice `abre: false`. Hoy es lo segundo. Se resuelve decidiendo en el capítulo 107, nunca saltándose el sello.
- **Falta `variant_trades.parquet`.** El lote no se cosechó con las operaciones de cada variante: vuelve al capítulo 18 y al 37.
- **«Sin dato».** Quedan menos de 30 variantes tras los filtros, o la madre no tiene operaciones fuera. Fabrica más variantes o revisa los suelos.
- **Cambió `criteria.yaml` y el sello se cerró otra vez.** El registro firma la regla: repite la calibración (capítulo 107).

107. Calibración del paso 17 — ¿cuántas veces dice PASA de la nada, y cuántas NO PASA de algo bueno?

Qué pregunta responde

El criterio del paso 17 (WFC) es una tabla: según el **orden** (positivo, neutro, perverso) y el **nivel** (¿gana fuera lo que elegiría un optimizador, y más de lo que daría el azar?), la madre sale **PASA, LIMBO o NO PASA**. El nivel lleva su propio nulo de volteo desde el 2026-10-07: sin él, una de cada dos familias sin ventaja salía PASA. Antes de dejar que esa tabla lea oos2 —el único tramo que nadie ha mirado— hay que saber cuánto se equivoca. Este comando lo mide con familias **fabricadas**, de las que se conoce la respuesta:

- **familias sin ventaja**: ¿cuántas salen PASA? Es el **falso PASA**;
- **familias con ventaja plantada**: ¿cuántas salen NO PASA? Es el **falso NO PASA**, el error que no se revisa (las que salen LIMBO quedan en espera, no se pierden).

Y escribe el **registro** que el sello de oos2 exige: sin registro, o con un registro que diga `abre: false`, ningún estudio del paso 17 puede abrir oos2, ni a mano.

Además pone al lado el **criterio anterior** (sin nulo en el nivel), calculado con los mismos sorteos, para ver con el coste junto a la ganancia lo que cambió la decisión del dueño.

Cuándo lo usas, y cuándo no

Lo usas una vez antes de la primera lectura de oos2 del paso 17, y otra vez **cada vez que cambie la entrada `"17"` de `pipeline/autopilot/criteria.yaml`**: el registro firma esa entrada, y si la cambias el registro viejo deja de valer y oos2 vuelve a quedar sellado.

No lo usas para juzgar una madre: no mira ninguna estrategia tuya. Y **no sirve para hacer pasar el criterio**: si una tasa supera su tope, el comando lo dice arriba y deja `abre: false`. Cambiar el criterio, o los topes, es decisión tuya.

Antes de empezar

Nada. No toca SQX, no lee oos2 y no necesita ningún lote. Si existen lotes con `variant_trades.parquet` (los escribe la fábrica del paso 16.5), los usa para el ensayo de la sección 4; si no hay ninguno lo dice y sigue.

Es un cálculo pesado: con una corrida de SQX en marcha, lánzalo con pocos procesos (`--workers 4`).

Cómo se ejecuta

```
python3 -m studies.optimisation.wfc.calibrate
```

flag	obligatorio	qué hace
`--workers N`	no	procesos en paralelo; por defecto 16
`--set clave=valor`	no	cambia una clave de `studies/optimisation/wfc/calibrate/config.yaml` para probar (p. ej. `--set families=50`). **Con `--set` el registro no se escribe**: el que abre oos2 sólo sale de la configuración tal como está

El comando vive en `studies/optimisation/wfc/calibrate/__main__.py`. Tarda unos 5 minutos con 16 procesos y la máquina libre (20 minutos con 4 procesos y una corrida de SQX al lado). No toca SQX.

Lo que se puede ajustar, en `calibrate/config.yaml`:

clave	valor	qué es
`families`	300	familias por escenario (ledger: `wfc.calibration.families`)
`max_false_pass`	0,40	tope del falso PASA (ledger; del Council, provisional)
`max_false_fail`	0,25	tope del falso NO PASA con ventaja plantada (ledger; del Council, provisional)
`family.variants`	300	variantes por familia
`family.years_in`	15	años de dentro (build + oos1)
`family.own_noise`	0,4	cuánto ruido propio lleva cada variante: cuanto menos, más se parecen entre sí
`grid.years_out`	1 · 2,75 · 5	años de la ventana de fuera
`grid.edges`	0,25 · 0,5 · 1,0	ventaja plantada, en Sharpe anualizado del exceso
`grid.costs`	−0,1 · −0,25 · −0,5	familias sin ventaja con media negativa (costes)
`reference`	2,75 años, Sharpe 0,5	las casillas de las que sale `abre`

Qué produce

- `AlgoData/ledger/wfc_calibration.json` — **el registro**. Lo lee el sello (`ledger.gate.seal`). Lleva `fecha`, `criteria_sha256` (la firma de la entrada `"17"` de `criteria.yaml`), `abre`, el motivo, los topes y todas las tasas. **Cada corrida sin `--set` lo sobrescribe.**
- `AlgoData/ledger/wfc_calibration.md` — el informe que se imprime, guardado.

Cómo se lee el resultado

```
$ python3 -m studies.optimisation.wfc.calibrate --workers 4
# Calibración del paso 17 (WFC)

**El registro ABRE el bloque ciego** (`abre: true`): las dos tasas de referencia están bajo su tope.

Criterio tal como está escrito (con nulo en el nivel), a 2,75 años fuera: falso PASA sin ventaja **0,02** (tope 0,40, ± 0,01; sin nulo en el nivel er…

## 1. Lo que decide el dueño: el criterio escrito frente al anterior, coste junto a ganancia

- **1,00 años fuera** — falso PASA sin ventaja: 0,49 sin nulo en el nivel, **0,04** con él; falso NO PASA con Sharpe 0,50: 0,30 sin nulo, **0,30** co…
- **2,75 años fuera** — falso PASA sin ventaja: 0,49 sin nulo en el nivel, **0,02** con él; falso NO PASA con Sharpe 0,50: 0,22 sin nulo, **0,22** co…
- **5,00 años fuera** — falso PASA sin ventaja: 0,47 sin nulo en el nivel, **0,03** con él; falso NO PASA con Sharpe 0,50: 0,12 sin nulo, **0,12** co…

| años fuera | familia | Sharpe | PASA | LIMBO | NO PASA | PASA (sin nulo) | LIMBO (sin nulo) | NO PASA (sin nulo) |
| --- | --- | --- | --- | --- | --- | --- | --- | --- |
| 1,00 | sin ventaja | 0,00 | 0,04 | 0,47 | 0,50 | 0,49 | 0,01 | 0,50 |
| 1,00 | sin ventaja | -0,10 | 0,03 | 0,40 | 0,57 | 0,40 | 0,03 | 0,57 |
```

La primera línea es el veredicto: si el registro abre o no el bloque ciego, y por qué. Se calcula **sólo con el criterio tal como está escrito** (con el nulo en el nivel), en dos tasas de referencia a 2,75 años fuera: el falso PASA sin ventaja y el falso NO PASA con Sharpe 0,5 (la peor de las familias con y sin orden).

Sección 1 — lo que decides. Para cada escenario, PASA / LIMBO / NO PASA de dos criterios:

- **El escrito** (columnas sin «sin nulo»): el top gana si su mediana fuera es mayor que cero **y** esa mediana está por encima de lo que da el azar. Se voltea el signo de cada día (el mismo para todas las variantes, como en el nulo del orden) y se mira en cuántos sorteos la mediana del top iguala o supera a la observada: ese es el **p del nivel**, y se pide p ≤ 0,05.
- **El anterior** (columnas «sin nulo»): sólo la mediana mayor que cero. Se deja como comparación.

En las filas «sin ventaja» la cifra que importa es el **PASA** (bueno: bajo). En las filas «ventaja», el **NO PASA** (bueno: bajo; el error irreversible); el LIMBO es una ventaja que queda en espera.

```
$ ...  (secciones 3 y 4)
## 3. ¿Se parecen las variantes como en un lote real?

Desviación de la ρ en el nulo de volteo, familias sin ventaja: mediana 0,23 (5 %–95 % de las familias: 0,05 a 0,43). La corrida de refer

| ruido propio | familia | Sharpe | sd de ρ en el nulo | PASA | NO PASA | PASA (sin nulo) | NO PASA (sin nulo) |
| --- | --- | --- | --- | --- | --- | --- | --- |
| 1,00 | sin ventaja | 0,00 | 0,08 | 0,03 | 0,45 | 0,52 | 0,45 |
| 1,00 | ventaja, sin orden | 0,50 | 0,07 | 0,31 | 0,09 | 0,89 | 0,09 |
| 1,00 | ventaja, con orden | 0,50 | 0,07 | 0,58 | 0,03 | 0,97 | 0,03 |
| 0,20 | sin ventaja | 0,00 | 0,48 | 0,02 | 0,48 | 0,49 | 0,48 |
| 0,20 | ventaja, sin orden | 0,50 | 0,50 | 0,24 | 0,19 | 0,78 | 0,19 |
| 0,20 | ventaja, con orden | 0,50 | 0,14 | 0,47 | 0,07 | 0,93 | 0,07 |

## 4. Ensayo en lotes reales, build → oos1 (tramos ya gastados; nunca oos2)

0 lotes: ningún lote tiene `variant_trades.parquet` todavía (lo escribe la fábrica del paso 16.5 en su próxima corrida en SQX).

## Cómo se lee
```

Sección 2 reparte cada escenario por las casillas de la tabla del criterio. **Sección 3** comprueba que las familias fabricadas se parecen entre sí como un lote real: la desviación de la ρ en el nulo debe caer en el rango medido en la corrida de referencia (0,09 a 0,41). **Sección 4** es el ensayo en lotes reales con build dentro y oos1 fuera; ahí no se sabe la verdad, así que sólo cuenta casillas.

Un ejemplo completo

La corrida del 2026-10-07 con el criterio nuevo (300 familias por escenario, 300 variantes, 2.000 sorteos, `--workers 4`, 20 minutos):

```
$ python3 -m studies.optimisation.wfc.calibrate --workers 4
# Calibración del paso 17 (WFC)

**El registro ABRE el bloque ciego** (`abre: true`): las dos tasas de referencia están bajo su tope.

Criterio tal como está escrito (con nulo en el nivel), a 2,75 años fuera: falso PASA sin ventaja
**0,02** (tope 0,40, ± 0,01; sin nulo en el nivel era 0,49) · falso NO PASA con Sharpe 0,50 **0,22**
(tope 0,25; sin nulo 0,22).
...
0 lotes: ningún lote tiene `variant_trades.parquet` todavía
-> /home/sergioguslw/Desktop/AlgoData/ledger/wfc_calibration.json
```

Cómo se lee, a 2,75 años (la ventana de oos2):

familia	PASA con nulo	PASA sin nulo	NO PASA (los dos)	LIMBO con nulo
sin ventaja, sin costes	0,02	0,49	0,48	0,50
sin ventaja, costes de Sharpe −0,25	0,04	0,31	0,65	0,31
ventaja 0,5 sin orden (meseta)	0,20	0,75	0,22	0,58
ventaja 0,5 con orden	0,43	0,91	0,09	0,49
ventaja 1,0 sin orden	0,51	0,89	0,07	0,42
ventaja 1,0 con orden	0,92	1,00	0,00	0,08

- **Sin el nulo en el nivel**, una de cada dos familias sin nada dentro salía PASA (0,49). Con él, 0,02: ruido y PASA casi no coinciden.

- **El coste**: el nulo no añade ningún NO PASA (un top que no gana es NO PASA en los dos criterios); lo que quita de PASA va a **LIMBO**. Una meseta de Sharpe 0,5 pasa de PASA 0,75 a 0,20 y el resto espera: una ventaja de Sharpe 0,5 en 2,75 años es casi indistinguible del azar.
- **Las dos tasas de referencia están bajo su tope** (falso PASA 0,02 ≤ 0,40; falso NO PASA 0,22 ≤ 0,25), así que el registro dice `abre: true`. **No es una promesa sobre las madres**: dice que la regla es medida. El falso NO PASA queda cerca de su tope (0,22 de 0,25): una meseta que de verdad gane poco sale NO PASA una de cada cinco veces.

Qué NO te dice

- **No dice que tus madres tengan ventaja ni que no.** Mide la regla, no las estrategias.
- **Sin el nulo en el nivel el falso PASA no bajaba con más familias ni con más variantes.** Una familia sin ventaja tiene la mediana del top por encima de cero la mitad de las veces, y el orden sólo aparta el 5 % que llama perverso: 0,5 × 0,95 ≈ 0,475. Es aritmética, no ruido de la simulación; por eso el nivel lleva ahora su nulo.
- **Las familias son fabricadas**: un factor común y ruido normal. Los lotes reales tienen colas más gordas y rachas; el ensayo de la sección 4 es lo que lo contrastará cuando haya lotes.
- **Una ventaja de Sharpe 0,5 en 2,75 años es casi indistinguible del azar** (estadístico t de 0,83). Ninguna regla sobre oos2 puede a la vez dejar fuera la nada y hacer pasar eso; el criterio nuevo la deja en LIMBO, y la sección 1 enseña cuánto cuesta cada elección.
- El ensayo de la sección 4 **no anota miradas en el ledger**: build y oos1 ya están gastados.

Si algo falla

- `ledger: oos2 sellado: la calibración registrada el … no abre el bloque (abre: false)` al lanzar el WFC: no es un fallo. El registro dice que una tasa supera su tope; decide tú si cambias el criterio o el tope, y vuelve a calibrar.
- `… no hay calibración registrada para este criteria.yaml`: la entrada `"17"` cambió desde la última calibración. Vuelve a lanzar este comando.
- **«Registro NO escrito»** en el informe: lo lanzaste con `--set`. Quítalo.

25. CSCV — ¿te fías de tu forma de elegir parámetros?

Qué pregunta responde

El estudio del capítulo 19 (WFC) te dice si **esta** superficie de parámetros aguantó fuera de muestra. Éste responde a la pregunta de debajo: **tu manera de elegir los parámetros, ¿escoge sistemáticamente los que van a decepcionar?**

Una correlación se mide sobre una sola partición, y esa frontera la elegiste tú. El CSCV (Bailey, Borwein, López de Prado y Zhu, «The Probability of Backtest Overfitting») parte la historia en **16 bloques** de longitud casi igual y prueba **las 12.870 formas** de usar ocho para elegir y ocho para juzgar. En cada una elige la variante con mejor Sharpe semanal dentro y mira qué puesto saca fuera. La especificación es la del dueño (`criterios-paso-18-cscv-propuesta`, 2026-10-07). **Puede descartar y nunca confirma: PASA significa «no descarta».**

Lo que mide, una línea cada cosa:

- **PBO**: la fracción de particiones en que lo elegido dentro queda por debajo de la mediana fuera.
- **P de pérdida**: la fracción de particiones en que lo elegido pierde dinero fuera. El PBO habla de puestos; esto habla de dinero.
- **Línea de degradación (β y R^2 ajustado)**: el Sharpe fuera de lo elegido contra su Sharpe dentro. β cerca de 1 = el número de dentro se conserva; cerca de 0 = no decía nada.
- **Dominancia estocástica**: si lo elegido sale mejor que el Sharpe fuera de TODAS las variantes en todas las particiones. En primer orden se admite una tolerancia (`dominance.tolerance`, 0,01) porque lo elegido es un subconjunto del total y la versión estricta falla por construcción justo bajo el máximo; la mayor brecha se anota. El segundo orden es estricto.
- **Recorte del Sharpe (HC)**: 1 − mediana fuera / mediana dentro. Sin dato si la mediana de dentro no es positiva.
- **Frecuencia de selección** y **lectura de la madre**: cuántas veces sale elegida cada variante, y qué le pasa a la madre en concreto.

La matriz de partida son **celdas semanales**: el exceso semanal de cada variante, calculado desde sus operaciones (`variant_trades.parquet`, por `engines.weekly.panel`), neto; y bruto, solo para «orden estable». Se guarda junto al lote como `cscv_weekly.parquet`.

El nulo. Todas las columnas se igualan (misma media) y se vuelven a sortear las semanas por bootstrap estacionario, las mismas filas para todas las variantes; se repite el CSCV entero por sorteo, en neto y en bruto (un quinto tipo de sorteo: 5 × `null.draws` CSCV por madre). Así el PBO real se compara con el PBO de una familia sin ventaja. **La potencia**: se planta un Sharpe anual δ en el 10 % de las columnas igualadas (nuevo en cada sorteo) y se mide qué parte de los sorteos deja el PBO bajo el p5 del nulo.

El estado

> **Desde el 2026-10-12 el paso 18 es una columna** (encargo 65): el CSCV y el PBO se calculan y se enseñan, pero `criteria.yaml` ya no les pone regla (`rules: []`), así que el autopilot no juzga ni corta con ellos. La tabla de abajo es el estado que sigue diciendo el estudio.

estado	cuándo
NO PASA	P de pérdida ≥ 0,5 (`state.p_perdida_fail`), o PBO ≥ el p95 de su propio nulo
LIMBO	P de pérdida por encima del p95 de la P de pérdida de su propio nulo (`p_perdida_sobre_p95`), o menos de 30 variantes tras los filtros (`panel.min_variants`)
PASA — no descarta	en cualquier otro caso; se añade «orden estable» cuando el PBO queda en o bajo el p5 del nulo en neto Y en bruto

Los cortes son filas del ledger (`ledger/thresholds.yaml`), no números del código. El PBO se muestra como porcentaje con un decimal; las fracciones crudas están en `cscv.json` .

Cuándo lo usas, y cuándo no

Lo usas después de fabricar y retestear las variantes de una madre (paso 18). El pipeline lo corre solo; a mano tiene sentido para reexaminar un lote ya recogido o varios a la vez.

No lo usas para saber si la estrategia va a ganar dinero el año que viene. El CSCV rompe la cronología a propósito: mezcla trozos de 2009 con trozos de 2021 y juzga el procedimiento, no el futuro. Lo que mira hacia delante es la walk forward matrix (capítulos 14 y 34).

No lo usas con pocas variantes: con menos de 30 tras los filtros queda en LIMBO.

Y no lo uses para separar un 45 % de un 55 %. Sobre paneles de puro ruido el PBO de un panel concreto se mueve mucho (medido: desviación típica de 0,21). Por eso el estado no lee el PBO suelto, sino contra el p95 de su propio nulo.

Antes de empezar

1. **Los costes del símbolo acordados**: `python3 -m core.assets <SÍMBOLO>` sale con código 0.
2. **El lote tiene** `variant_trades.parquet` (las operaciones de cada variante, que cosecha `/variants` , capítulo 18). Sin él el comando se para y dice qué hacer. **Hoy ningún lote del directorio de datos lo tiene**, así que nada basado en operaciones se ha corrido sobre una madre real.

No toca SQX ni arranca nada: es lectura de ficheros y unos segundos de Python. Se puede lanzar con la GUI abierta.

Cómo se ejecuta

```
python3 -m studies.optimisation.cscv.report --work <lote> [<lote> ...] --family <familia> [--set k=v]
```

flag	obligatorio	qué hace
`--work`	sí	**uno o varios** lotes (carpetas). Varios comparten una sola cola de procesos; cada uno escribe en su propio `estudios/`
`--family`	sí	la familia de plantillas, tercera parte del estudio del ledger. Símbolo y timeframe salen del lote
`--set k=v`	no	cambia un mando de `config.yaml` solo en esta corrida

La antigua `--blocks` ya no existe: los 16 bloques son la fila `cscv.blocks` del ledger. El comando pregunta a la puerta del ledger y apunta su mirada; el CSCV lee la historia entera (`build` , `oos1` y `oos2`), así que `CSCV` está en la reserva de `oos2` de `assets/_policy.yaml` (decisión del 2026-09-27). **No uses un lote real para comprobar un cambio en el CSCV**: para eso está `tests/test_cscv.py` .

Cuánto tarda (medido el 2026-10-07, 48 núcleos físicos, máquina cargada):

variantes	sorteos	reloj	CPU
300	200	3,5 s	72 s
1.000	200	8,9 s	243 s
300	1.000	13,5 s	349 s
1.000	1.000	40,5 s	1.195 s

784 semanas; como mucho 107 MB por proceso. Los sorteos por defecto son 200.

Qué produce

En `<lote>/estudios/`:

archivo	qué es
`cscv.html`	**el informe; es lo que se mira**, con las cuatro pestañas y el veredicto
`cscv.md`	lo mismo en texto
`cscv.json`	pequeño: el contrato, los `facts` (los que lee el piloto) y el `rendimiento`
`cscv_particiones.parquet`	una fila por partición; el detalle que nadie carga para decidir
`cscv_sorteos.parquet`	una fila por sorteo del nulo

Se sobrescriben al volver a correr. Junto al lote queda `cscv_weekly.parquet`, la caché de celdas.

Cómo se lee el resultado

Sin captura: la salida nueva solo se ha producido con datos sintéticos (no hay un lote con `variant_trades.parquet`), y las capturas y salidas de terminal anteriores eran del CSCV diario con tres reglas de selección, que ya no existe.

1. **El estado** (tabla de arriba) y su razón. Es lo que cuenta.
2. **PBO frente a su nulo**: el PBO real, y el p5/p95 de la familia sin ventaja. `pbo_p` es la cola baja (fracción de sorteos del nulo en o bajo el PBO real, contando el real); `pbo_p_mas` la alta.
3. **P de pérdida**: de verdad dinero. Es la que lleva a NO PASA o LIMBO por sí sola.
4. **β y R² ajustado** de la línea de degradación, y la **dominancia** con su mayor brecha (`dominancia.brecha`).
5. **HC, frecuencia de selección y la madre**: cuánto del Sharpe se pierde al pasar fuera y qué hace la madre en concreto.
6. **Potencia**: qué parte de los sorteos con un Sharpe δ plantado habría dejado el PBO bajo el p5. Si es baja, un PASA dice poco.
7. **Avisos**: la mediana del PBO del nulo fuera de banda (el remuestreo falla), duplicados exactos quitados (su proporción se anota) y los controles fuera; la madre entra siempre.

Un ejemplo completo

```
python3 -m studies.optimisation.cscv.report \
  --work ~/Desktop/AlgoData/pipeline/XAUUSD/Strategy_17-9-39
~/Desktop/AlgoData/pipeline/XAUUSD/Strategy_10-1-79 \
  --family <familia>
```

Lee las dos madres, corre la cola una vez y deja, en el `estudios/` de cada una, los cinco ficheros de arriba. Con datos reales el primer ejemplo será el primer lote que tenga `variant_trades.parquet`.

Preguntas abiertas del dueño

> **La media «m» común del nulo.** Hoy cada columna igualada recibe la **mediana del Sharpe semanal del panel**. Alternativas: cero (solo el balancín: la P de pérdida del nulo ronda 1 sin remuestreo y 0,5 con él) o la mediana de las medias crudas con desviación común.
>
> **El corte de LIMBO de la P de pérdida.** El corte fijo de 0,25 se quitó (dueño, 2026-10-08): quedaba por debajo de la mediana del nulo (0,36 a 0,38). Ahora es LIMBO si la P de pérdida supera el p95 de la P de pérdida de su propio nulo. Medido: el nulo se centra en la mediana del propio panel, así que la regla salta en torno al 5 % con columnas independientes, pero casi nunca con variantes correlacionadas (0,5 % con correlación 0,2; 0 % desde 0,6): en lotes reales LIMBO casi no sale por aquí. Y el corte de NO PASA (0,5) suspende al 29 % de los paneles correlados sin ventaja con Sharpe semanal 0,03, y al 51 % con 0.
>
> **Dominancia de primer orden con tolerancia**, frente a quitarla y dar solo la brecha. **Números aleatorios comunes** entre el nulo y las potencias (menos ruido al mismo coste: no se pidió) y **más sorteos** (1.000 sorteos con 1.000 variantes cuestan 40 s): el defecto sigue en 200.

Qué NO te dice

- **No dice si la estrategia funcionará hacia delante.** Baraja la historia a propósito.
- **No dice que la estrategia sea buena.** Un PBO bajo sobre una superficie donde todo pierde significa que eliges de forma fiable dentro de un conjunto malo. Por eso está la P de pérdida.
- **PASA es «no descarta», no «confirma».** Un panel sin potencia para ver una ventaja pasa igual.
- **No distingue un PBO del 45 % de uno del 55 %** por sí solo.

Si algo falla

- **El comando se para pidiendo `variant_trades.parquet`.** El lote no se cosechó con las operaciones de cada variante: vuelve a `/variants` (capítulo 18).
- **`the following arguments are required: --family`.** Pásale la familia de plantillas del lote.
- **Sale LIMBO con muchas variantes.** Mira primero `nulo.p_perdida_p95` y el número de variantes (`panel.min_variants`).
- **Salen muchas menos variantes de las que reteseaste.** El custodio no volcó su databank a disco antes de apagarse; comprueba `n` contra `n_returned` de `ran.json`.

39. La nube de parámetros — ¿qué parámetros mueven el resultado, y se puede leer ese ranking?

Qué pregunta responde

Cuando la fábrica de variantes (`09-optimizacion.pdf › 18-variantes`) escribe un lote de versiones de una estrategia madre y el custodio las retestea, lo que queda en disco es **la superficie** de lo que esa lógica rinde según cómo la ajustes. Este comando la lee y responde dos cosas:

1. **¿Quién mueve el resultado?** Qué parámetros deciden el exceso por año y cuáles son decorativos.
2. **¿Ese ranking se puede leer, o es ruido?** Una superficie que no explica nada reparte igualmente un ranking; el comando dice si la que ha ajustado explica más de lo que explicaría el azar.

La medida es el **exceso por año**, la misma del paso 14: lo que cada variante gana menos lo que el mercado le habría pagado solo por estar dentro, en dinero de la cuenta por año.

Cuándo lo usas, y cuándo no

Lo usas al final del paso 16.5, cuando el lote ya tiene `variant_trades.parquet` . No habla con SQX: puedes lanzarlo con cualquier instalación abierta o cerrada. En el flujo normal no lo lanzas a mano: el autopilot lo lanza al final del 16.5 para cada madre.

Es un diagnóstico, no una regla. No corta nada (`criteria.yaml` , paso «16.5», `cut: null`), no elige una variante mejor y no aprueba ni descarta la madre. Si algún día quieres mover los parámetros hacia donde la superficie parece mejor, lo decides tú y se revalida con datos que este estudio no ha mirado.

No sirve para saber si la madre es un pico (eso lo juzga el paso 14), ni si elegir por el pasado funciona (el WFC, `09-optimizacion.pdf › 19-wfc`), ni cómo se sostiene en el tiempo (la WFM), ni qué pasa en otros mercados (superficies multimercado, paso 18.5).

Antes de empezar

Hace falta el directorio del lote con su `variant_trades.parquet` (lo escribe la cosecha de `09-optimizacion.pdf › 18-variantes`). **Hoy no existe ningún lote real con ese fichero** (pendiente en OPEN.md, parte V): por eso este capítulo no enseña ningún resultado real, sólo el comando.

Nunca lee `oos2` , lo lance quien lo lance: pide al fichero sólo los tramos `build` y `oos1` , y el filtro se aplica antes de cargar ninguna fila. Y sólo el mercado principal (Main).

Cómo se ejecuta

```
python3 -m studies.optimisation.cloud.report --work ~/Desktop/AlgoData/pipeline/XAUUSD/<lote>
```

flag	obligatorio	qué hace
`--work`	sí	el directorio del lote, con su `variant_trades.parquet`
`--out`	no	escribe además todos los números en un JSON
`--set`	no	cambia un ajuste sin editar el YAML, `--set clave=valor`

Es aritmética sobre una superficie: segundos, sin CPU del worker.

```
$ python3 -m studies.optimisation.cloud.report --help
usage: report.py [-h] --work WORK [--out OUT]
                 [--set OVERRIDES [OVERRIDES ...]]

The command: which parameters move a mother's excess across her variant batch,
and whether the ranking can be read.

options:
  -h, --help            show this help message and exit
  --work WORK           batch directory, with its variant_trades.parquet
  --out OUT             also write every number here as JSON
  --set OVERRIDES [OVERRIDES ...]
```

Lo que ves arriba es la ayuda del comando (`--help`), no un resultado: todavía no hay un lote real con el que sacar uno.

Qué produce

En la carpeta `estudios/` del lote: `cloud.json`, `cloud.html` y `cloud.md`; con `--out`, además el resumen solo. No toca nada más. La ventana pinta lo mismo desde esos datos.

Para el autopilot deja estos **hechos** en `cloud.json`: `vivos` (parámetros vivos), `coeficientes`, `variantes.build` y `variantes.oos1` (variantes en el mapa, y de ellas las que tienen alguna operación en oos1) y, por tramo (`build`, `oos1`): `r2_ajustado`, `r2_barajado`, `legible`, `sobol_primer.<parámetro>` y `sobol_total.<parámetro>`.

Cómo se lee el resultado

Qué variantes entran. De lo que vuelve del retest salen: los controles (canarios y pares inertes); las variantes que operan menos del suelo de **15 operaciones por parámetro vivo** en build (si la madre misma no llega, se avisa y el suelo no se aplica); y los duplicados exactos (mismo beneficio al céntimo en build y en oos1). La madre nunca se descarta. La pestaña «Qué variantes entran» cuenta cuántas quita cada filtro.

La superficie. Sobre los parámetros que de verdad varían en el lote (los «vivos») se ajusta una superficie cuadrática: cada parámetro, su cuadrado y sus cruces. El mapa principal es el de build; el de oos1 es la misma superficie reajustada, como lectura secundaria para ver si los parámetros que mandan en build siguen mandando después.

Los índices de Sobol. Cada parámetro recibe dos números de 0 a 1. *Primer orden*: cuánto de la variación de la superficie produce él solo. *Total*: eso más lo que hace combinado con los demás. Un total muy por encima del primer orden indica interacción; si los totales suman mucho más de 1, el resultado depende de combinaciones finas.

¿Se puede leer? R^2 ajustado contra su referencia barajada. El R^2 ajustado dice qué parte de las diferencias entre variantes explica la superficie, descontando los coeficientes que gasta. Se compara con lo que da la **misma superficie sobre el exceso barajado al azar** entre las variantes (200 barajados, percentil 95). El ranking es **«legible» sólo si el real supera esa referencia**. Si no, el informe avisa **«Sin estructura distinguible del ruido»** y el ranking no se lee.

Mapas de calor. Para los parámetros con mayor índice total (`heatmap_pairs`, 4 parámetros, que dan 6 mapas) se cruzan de dos en dos: cada mapa divide los dos ejes en 6 tramos iguales y pinta la **mediana del exceso por celda**, con la madre marcada y un solo rango de color para todos.

Avisos. Además del de ruido: menos de 8 variantes por coeficiente («pocas variantes»), el suelo no aplicado a la madre, y parámetros que el suelo deja con un solo valor («colapsados», fuera de la superficie). Con menos variantes que coeficientes no hay mapa que ajustar: el informe lo dice, no falla.

Un ejemplo completo

Todavía no existe un lote real con `variant_trades.parquet`, así que no hay salida que pegar sin inventarla. Cuando exista, la secuencia es esta:

1. `python3 -m studies.optimisation.cloud.report --work <lote>`.
2. Mira primero «¿Se puede leer?» de la pestaña build: R^2 ajustado frente a la referencia barajada.
3. Si es legible, lee la tabla de Sobol (quién manda) y los mapas de calor de los parámetros de arriba.
4. Si sale «Sin estructura distinguible del ruido», no leas el ranking: no hay nada que leer.

Qué NO te dice

- **No decide nada.** Sin regla, sin corte, sin variante recomendada.
- **Un ranking ilegible sigue pareciendo un ranking.** Una superficie ajustada a ruido puro reparte igualmente índices de Sobol (los de primer orden suman alrededor de 0,8). Por eso se mira antes la referencia barajada. Ojo: con unas 458 variantes y 105 coeficientes, la referencia del R^2 **ajustado** ronda 0,06; el 0,23 que se ve en otros sitios es el R^2 sin ajustar. Y el ajuste usa el rango real de la matriz del diseño: un parámetro con dos valores hace que su cuadrado sea copia de sí mismo y no cuenta como coeficiente nuevo.
- **Legible no es robusto.** Dice que hay forma que leer, no que la madre sea buena ni que esa forma se mantenga fuera de los datos usados.
- **Nada sobre `oos2`, otros mercados ni la evolución en el tiempo.**
- **Los índices describen la superficie ajustada**, no los backtests: valen lo que valga su R^2 ajustado.

Si algo falla

- **Falta `variant_trades.parquet`:** el lote es de diseño antiguo o no se ha cosechado. Un lote del diseño estratificado antiguo no se puede leer aquí.
- **«pocas variantes» o «no hay mapa que ajustar»:** el lote devolvió menos variantes de las que pide la superficie. Se arregla en el tamaño del lote, no aquí.
- **Parámetros «colapsados»:** tras quitar las variantes que apenas operan, ese parámetro sólo tiene un valor. No bajes el suelo para taparlo: la rejilla está llena de combinaciones que no operan.

52. Superficies por mercado — ¿lo que va mejor en el mercado principal va mejor en los demás?

> **Condicionado por madre desde el 2026-10-12** (encargo 65, fila 8): el autopilot lo corre en cada madre solo si el `descuadre_sqx` que dejó el paso 17 (`estudios/wfc.json` , el mayor de los tramos) es menor que `wfc.sqx_gap_relative` . Si no lo es, o falta la cifra, lo salta con una línea («18.5 saltado en : descuadre con SQX 0,46 ≥ 0,01 — el repreciador no reproduce a SQX en esta madre»); esa madre queda en LIMBO en este paso, nunca cortada, y la cola sigue. Ya no hay lectura previa sin oos2, y el paso 20 aún no lo lee. El estudio se puede lanzar a mano.

Paso **18.5** del workflow, reescrito el 2026-10-07 desde la propuesta del dueño (`criterios-paso-18.5-superficies-propuesta.md`). Lee el lote de variantes del paso 16.5, no habla con SQX y no corre ningún backtest.

Qué pregunta responde

Que una estrategia gane en otro mercado es poca prueba. Que **el orden de sus variantes** viaje de un mercado a otro es mucha más, porque eso no lo produce la suerte de un backtest.

Para cada mercado declarado *m* en `assets/_markets.yaml` (no son 9 fijos: los mercados dependen del activo, y las pruebas sintéticas del estudio usan 11):

- **rho_m**: la correlación de rangos de Spearman entre el exceso bruto de cada variante en el mercado principal y en *m*, sobre las variantes comunes. 1 = mismo orden, 0 = nada que ver, −1 = al revés.
- **rho_bar**: la media de los rho_m ponderada por peso. **Es lo que decide.**
- **Exceso**: lo que gana cada variante menos lo que el mercado paga por estar dentro (la deriva del gate), por semana, con los costes de cada mercado. El **bruto** (antes de costes) ordena las variantes; el **neto** sirve para la prima.

Cuándo lo usas, y cuándo no

Lo usas en el paso 18.5, junto al WFC (17) y el CSCV (18): consume el mismo lote del 16.5. Es gratis: no toca SQX.

No lo usas para elegir una variante, ni para decidir que el edge funciona en otro mercado (eso es el crossmarket, `07-otros-mercados-y-timeframes.pdf › 39-crossmarket-lote`): aquí solo se pregunta si **el orden** viaja.

Dos lecturas, un solo comando

lectura	mercado principal	otros mercados	cuándo	qué puede hacer
decisoria	build + oos1	build + oos1	siempre	`perverso` → NO PASA; `sin_dato` → LIMBO
oos2	build + oos1	oos2	solo con `--oos2` (bloque ciego)	`perverso` → LIMBO, nada más

Sin `--oos2` ni se pide a la puerta ni se lee ningún tramo `oos2` , y no se escribe ningún hecho `oos2.*` . El `oos2` del mercado principal no lo lee ninguna de las dos. El piloto corre primero la decisoria (antes del bloque ciego) y la juzga; luego, dentro del bloque, vuelve a calcular con `--oos2` , y esa segunda lectura **solo puede añadir LIMBO**.

Antes de empezar

Un lote de variantes retesteado **con los cross-checks de mercado** y con `variant_trades.parquet`. SQX puede estar abierto o cerrado: no se le habla. Los costes de cada mercado se leen de su ficha (`reprice.costs`).

Cómo se ejecuta

```
python3 -m studies.optimisation.marketSurfaces.report --work <lote> --family <f> [--oos2] [--out DIR]
[--set a.b=v]
```

flag	obligatorio	qué hace
`--work`	sí	la carpeta del lote de variantes
`--family`	sí	la familia de plantillas del lote; con el activo y el timeframe forma el estudio del ledger
`--oos2`	no	añade la lectura de `oos2` (solo en el bloque ciego); sin él, solo build + oos1
`--out`	no	dónde escribir; si no lo pones, `<work>/estudios/`
`--set a.b=v`	no	cambia un ajuste de `config.yaml` solo en esta corrida

Medido solo con paneles sintéticos (`tests/test_marketsurfaces.py`): 500 variantes × 11 mercados × 620 semanas × 2.000 sorteos tardan 18 s (28 s con `--oos2`), sin contar la lectura.

Qué produce

En `<lote>/estudios/`:

fichero	qué es
`marketSurfaces.json`	pequeño: sobre, resumen, `facts` y cuatro pestañas ligeras; de él salen `.html` y `.md`
`marketSurfaces.html` / `.md`	el informe que se lee
`marketSurfaces_detalle.json`	las rejillas; no lo carga quien decide
`mercados.csv`	una fila por mercado: su rho, sus variantes comunes, su peso
`verdict.csv`	la llamada de la madre

`summary.segundos` es lo que tardó la corrida.

Qué se mide, y qué decide

- **Variantes usables en el principal**: no duplicadas (por huella de sus operaciones `Main` en build + oos1, la madre primero) y por encima de los dos suelos de operaciones del panel. El **conjunto común** de *m* son las que además llegan al suelo de operaciones fuera por los años que *m* tiene datos.
- **Un mercado entra en la media** con al menos 50 variantes comunes (`markets.min_common`) y como mucho la mitad empatadas (`markets.max_ties`; empatada = otra variante con exactamente su exceso bruto).
- **Pesos**: 1, o 1 repartido entre los miembros de un grupo `near_identical` que entran (el trío de índices USA cuenta 1).
- **El nulo**: un signo por semana, el mismo para todas las variantes y todos los demás mercados; el principal nunca se voltea. `p+` y `p−` son sus colas: la fracción de sorteos con un rho_bar igual o mayor / igual o menor que el real. Con corte 0,05: `p+ ≤ 0,05` = orden **positivo**, `p− ≤ 0,05` = **perverso**, el resto **neutro**; **sin dato** si no hay mercados suficientes.
- **Prima**: el 20 % de arriba de las variantes usables por exceso NETO en el principal; por mercado, (mediana del top − mediana de todas) / rango intercuartílico de su conjunto común; media ponderada contra el mismo nulo. En o bajo el percentil 5 del nulo (`premium.percentile`) → LIMBO.

El estado

estado	cuándo
NO PASA	orden decisorio **perverso**
LIMBO	sin dato; o prima baja (en o bajo el p5 de su nulo); o `oos2` perverso (solo con `--oos2`)
PASA	en otro caso: «la zona viaja» si el orden es positivo, «no descarta» si es neutro

Solo anotado, no decide

- **Referencia**: el rho del principal entre su build y su oos1 (`auto.rho`); el cociente rho_bar / rho_auto solo se escribe si ese rho supera `reference.min_rho`.
- **Potencia**: para cada rho *de* `power.rhos` *(* `potencia.r01`, `r02`, `r03`*), se simulan mercados sin ventaja mezclados con el principal para que su rho verdadero sea rho*, y se mide qué parte de los sorteos lee el nulo real como positivo. Dice si el estudio habría visto un orden que existiera.
- **Fin de semana**: si más del 10 % de las operaciones de los otros mercados cierran un lunes o martes tras un fin de semana abierto, se repite el orden decisorio con bloques de 2 semanas (`orden.estado_2sem`, un aviso; no cambia el estado).
- Los `facts` completos: `orden.{estado, rho, p_mas, p_menos, nulo_p5, nulo_p95, mercados, n_mercados, finde, estado_2sem}`, `prima.{valor, nulo_p5, bajo_p5}`, `auto.{rho, cociente}`, `variantes`, `duplicados`, `costes.descuadre`, y con `--oos2` `oos2.{rho, p_mas, p_menos, mercados, perverso}`. Un `oos2` sin datos escribe `oos2.perverso = 0`: se corrió y no encontró nada perverso.

Cómo se lee el resultado

Sin capturas: la salida nueva solo se ha producido con paneles sintéticos; ningún lote del directorio de datos tiene `variant_trades.parquet`*. Las capturas anteriores (pares de mercados, Jaccard, mapa de consenso) eran del estudio antiguo y ya no describen nada de lo que sale.*

1. **El estado** y su razón (tabla de arriba).
2. **`orden`**: rho_bar, `p+` / `p−` y la banda del nulo (p5–p95). Un rho_bar alto con el nulo ancho no es positivo.
3. **`mercados.csv`**: qué mercados entraron y cuáles quedaron fuera por pocas variantes comunes o demasiados empates. Cuantos menos entren, menos vale la media.
4. **Prima** frente a su nulo: si es baja, el orden viaja pero lo mejor no se distingue del resto.
5. **Potencia**: si es baja, un orden neutro no quiere decir que no exista.

Preguntas abiertas del dueño

> 1. **«Menos de 5 mercados (con pesos)»** se lee como la SUMA DE PESOS por debajo de `markets.min_weight` (5), no como el número de mercados.
> 2. **La familia plantada de la potencia** es la mezcla gaussiana descrita arriba; la otra lectura sería añadir una inclinación semanal a los paneles, y necesita una escala por mercado.
> 3. **El top** es un único conjunto, elegido por exceso neto en el principal y luego cruzado con el común de cada mercado; no se reelige por conjunto común ni por bruto.
> 4. **Los empates** son igualdad exacta del exceso bruto.
> 5. **Los duplicados** se deciden solo con las operaciones build + oos1 del principal, para que la lectura decisoria nunca abra `oos2`.
> 6. No se aplica el escalado por año de la propuesta: un rho y una prima escalada por el rango intercuartílico no se mueven con él.

Qué NO te dice

- **Si el edge paga en otro mercado.** Solo si el orden de las variantes viaja.
- **Que la zona sea buena**: PASA en neutro es «no descarta». Con poca potencia, un neutro no es evidencia de nada.
- **Nada medido con datos reales**: el código está comprobado solo con paneles sintéticos.
- **Que `oos2` confirme**: su lectura solo puede añadir LIMBO.

Si algo falla

- **El comando se para pidiendo `variant_trades.parquet`.** El lote no se cosechó con las operaciones de cada variante: `/variants` (capítulo 18).
- **`PermissionError: ledger: el paso 18.5 no puede mirar oos2`** (con `--oos2`): el activo no reserva `oos2` para `MarketSurfaces` en `assets/_policy.yaml`. No se ha leído nada.
- **Estado LIMBO sin dato**: menos mercados utilizables que el mínimo, o los comunes no llegan al suelo. Revisa `mercados.csv`.

34. Configurar la Walk Forward Matrix en SQX

> **Cambio del 2026-09-28.** La regla de «no quemar el oos2» ya no se aplica a humanos: puedes mirar el `oos2` y los pasos 17, 18 y 19 cuando quieras, desde la ventana o el terminal. Sólo un agente autónomo que decida solo (`ALGO_AUTONOMOUS=1`) sigue encontrando la puerta cerrada. El ledger sigue contando cada mirada.
>
> **Ojo con el nombre.** Esta página es la mitad de SQX del paso 19: **dejar la tarea `WFM` configurada** dentro de un custom project. Sacar lo que produce es `09-optimizacion.pdf › 09-wfm`, y el análisis que **juzga** (PASA / LIMBO / NO PASA, solo con las operaciones que cierran en `oos2`) es `09-optimizacion.pdf › 14-walkforwardmatrix`. El criterio de SQX de abajo **marca, no juzga**.

Qué pregunta responde

La Walk Forward Matrix reoptimiza la estrategia sobre una ventana que se desliza y corre cada elección sobre el tramo siguiente, **30 veces con particiones distintas**: 5 porcentajes de fuera de muestra (20 a 40 %) por 6 números de pasadas (6 a 16). La pregunta es si lo que optimiza bien predice lo que va bien después, y las 30 celdas existen para que la respuesta no dependa de haber elegido una partición con suerte.

Acaba en `oos2`

La ventana es `build..oos2`, o sea todo el histórico: el walk-forward necesita histórico delante para optimizar antes de correr nada. Pero por eso el 67–79 % del fuera de muestra de cada celda son datos que `build` y el paso 8 ya usaron: **el análisis no juzga por la celda entera, sino solo por las operaciones que cierran en `oos2`**, recargadas a los costes de `oos2` (capítulo 14).

Cuándo lo usas, y cuándo no

Úsalo cuando quedan pocos supervivientes y has hecho ya el 16.5 a 18.5 (variantes, WFC, CSCV, superficies). El 19 se juzga solo, con su propio estado; el paso 20 lee los cuatro a la vez.

No lo uses para "ver qué sale" sobre una población grande. No es una cuestión de CPU: es que mirar gasta la muestra, y no hay otra.

La rejilla que escribe

ajuste	valor	de dónde sale
ventana	`build..oos2` — 2008.01.01 → 2026.08.30 en XAUUSD	`wfm.segment`
a qué precio se cobra	al tramo **`oos1`**, el intermedio — no al último de la ventana	`wfm.costs_segment`
`Param1` — % de cada pasada fuera de muestra	20 → 40 de 5 en 5 (5 columnas)	`wfm.oos_pct`
`Param2` — número de pasadas	6 → 16 de 2 en 2 (6 filas)	`wfm.runs`
celdas	**30**	el producto de los dos
`MaxTests`	5.000 por paso y celda	`wfm.max_tests`
`±distribution` y el paso	±35 %, un 4 % por paso → `maxSteps=18`	`wfm.distribution_pct`, `wfm.step_pct`
tipo de periodo	`percent` (=10) — **fijo**: los periodos van por nº de pasadas y % de OOS, nunca por días ni barras	`wfm.period_type`
tipo de optimización	`floating` (=15) — **fijo**: la ventana se desliza manteniendo su longitud	`wfm.optimization_type`
fidelidad del walk-forward	`exact_is_exact_oos` (=2) — "Exact IS, Exact OOS"	`wfm.wf_type`
qué se permuta	los parámetros **recomendados** por SQX, nada más	fijo
precisión	2 (1 minuto)	`wfm.precision`, igual que la doctrina

La ventana empieza en `build` a propósito: un walk-forward necesita histórico DELANTE del tramo reservado, porque la primera pasada tiene que optimizar sobre algo antes de poder correr sobre nada. Lo que se gasta es el final, `oos2`.

El cobro es la excepción del proyecto. Los demás spans se cobran al último tramo (el precio pesimista); éste se cobra a `oos1`, el intermedio, por decisión del dueño (2026-09-24): son 18,7 años de los que `oos2` es la cola de 3,7, y cobrar quince años al precio de cuatro no describe nada. Hoy `oos1` y `oos2` comparten `spread_oos` —10 puntos y slippage 5 en XAUUSD— así que el número sale idéntico; la línea decide el día que el activo declare un spread propio para `oos2`. Un intermedio de verdad, entre los 5 de `is` y los 10 de `oos`, habría que declararlo en `assets/symbols/<SIM>.yaml`: esa cifra la pone el dueño, no el catálogo.

La fidelidad es el atributo `type` del elemento `<WalkForward>`, que SQX llama "Walk-Forward type" y que aquí va en `exact_is_exact_oos` (decisión del dueño, 2026-09-24). Lo que cambia respecto al default de SQX no son los decimales: es **quién elige los parámetros de cada paso**. Con `simulated IS`, el ganador de cada paso sale de recortar a su ventana una única tanda de backtests sobre el rango entero —arrastrando el camino de la equity, que con sizing por ATR no es neutro, y los trades abiertos antes del borde—; con `exact`, cada paso hace su propia optimización sobre su ventana, que es lo que harías en vivo al reoptimizar. La pregunta de la WFM es si la optimización **predice** el tramo siguiente, y con IS simulado la "optimización" es un recorte.

No es territorio nuevo: la WFM que ya corrió el maestro llevaba `type="2"` con `MaxTests` 500 sobre cinco años, y terminó.

Y por eso `MaxTests` bajó de 15.000 a 5.000. Los dos mandos compran cosas distintas: `MaxTests` compra una elección mejor por paso y **nada que se pueda leer después** —SQX guarda sólo el ganador de cada paso, no la población que probó (medido, `knowhow/sqx-format/wfm-in-settings-xml.md`)—, mientras que `wf_type` cambia lo

que significa el número que sí se guarda. Habiendo que elegir dónde van los núcleos, van a la exactitud. Cuenta la factura antes de lanzar: son 5.000 backtests por paso, por los 6–16 pasos de cada celda, por 30 celdas, por estrategia, a precisión 2.

El criterio: cuándo aprueba una casilla y cuándo aprueba la estrategia

Esto es lo que la tarea del maestro NO tenía —corría con el elemento de condiciones vacío, y con cero condiciones SQX aprueba todas las casillas con un 100— y es lo que ahora escribe el comando. Decodificado del propio SQX el 2026-09-24 (`knowhow/conditions/wfm-acceptance.md` tiene la disección):

```
score de la casilla = round(condiciones cumplidas / condiciones activas * 100)
casilla aprobada    = score >= threshold_pct          (80 %)
estrategia aprobada = hay un rectángulo de 4x4 casillas, en alguna de sus 6 posiciones,
                      con 12 o más aprobadas          (grid_passing_size, min_squares)
```

Las diez condiciones, agrupadas por la pregunta que hacen:

pregunta	condición	la cumplen
¿gana dinero en lo que no vio?	`WF Net profit (OOS) > 0`	66 %
	`WF Profit factor (OOS) > 1,05`	45 %
¿se parece el fuera de muestra a lo que prometía la optimización?	`Stability Profit factor > 60 %`	80 %
	`Stability Net profit > 20 %`	30 %
	`Stability Max DD % < 150 %`	79 %
¿reoptimizar conserva la forma del original?	`Score Profit factor > 85 %`	83 %
¿está repartido el resultado entre pasadas?	`% de pasadas rentables > 50`	51 %
	`máximo beneficio en una pasada < 50 % del total`	11 %
	`mínimo de trades en una pasada > 20`	55 %
	`máximo DD % en una pasada <= 25`	100 %

La columna *"la cumplen"* es el porcentaje de las **150 casillas reales** del databank `WFM` del maestro (5 estrategias x 30 celdas) que pasan ese umbral: es la calibración, y está ahí para que se vea qué condición separa y cuál no. Con este juego y `threshold_pct: 80`, 2 de esas 5 estrategias encuentran el área 4x4 con 12 y 3 no.

⚠ **Los defaults de SQX habrían tirado todo.** Su `Stability Net profit > 60` lo cumple el 1 % de esas casillas —la mediana está en 8,6 %, porque el tramo de optimización es justo el que se eligió por ser el mejor— y su `% de pasadas rentables > 70`, el 9 %.

El criterio marca, no borra (comprobado el 2026-09-26). Es una marca, nunca el juez: el análisis juzga en `oos2` (capítulo 14). El comando deja `DeleteFailedStrategies` en `false`, así que la estrategia que no encuentra el área **se queda** en el databank, marcada por SQX: `Cross Check filter in 'Walk-Forward Matrix': Robustness score didn't pass.` El export lo guarda (`status.parquet`, y `failed_in_sqx` en el manifest) y el análisis lo muestra como aviso **"SQX la marcó como FAILED"** en la ficha de esa estrategia. El veredicto se toma en Python, como en el resto del proyecto. El mínimo de 12 casillas se queda (dueño, 2026-09-26).

Los umbrales se tocan en `assets/_build.yaml`, bloque `wfm.conditions`. Cada línea es `{read, metric, op, value}`, donde `read` elige de dónde sale el número: `oos` (las pasadas fuera de muestra concatenadas), `stability` (OOS contra optimización, en %), `score` (la WF contra el backtest original, en %) o `special` (las cuatro columnas que SQX calcula pasada a pasada).

Antes de empezar

- El proyecto tiene que ser **custom** y llevar la tarea titulada `WFM`, que viene del donante congelado. Si no la lleva, el comando se niega en vez de fabricarla: el bloque y sus condiciones vienen de una tarea que escribió SQX.
- El `.cfx` **no puede estar abierto** por una instancia (regla dura 4). El comando lo comprueba.
- Costes pactados: `python3 -m core.assets <SIMBOLO>` (regla dura 5).
- La ventana `oos2` del activo tiene que estar decidida en `assets/_policy.yaml`. En los activos con `from: null` no hay ventana y el comando revienta — que es lo correcto.

No toca SQX ni ningún worker: reescribe un fichero y tarda menos de un segundo.

Cómo se ejecuta

```
python3 -m core.assets XAUUSD                       # preflight, BLOQUEANTE
python3 -m sqx.projects.wfm XAUUSD \
    --cfx ~/Desktop/SQX_w2/user/projects/XAUUSD_estudio/project.cfx \
    --input "SPP OOS"
```

flag	obligatorio	qué hace
`<SIMBOLO>`	sí	el activo, para sus costes y su ventana `oos2`
`--cfx`	sí	el `project.cfx` del custom project
`--input`	sí	el databank que lee: los supervivientes que han llegado al paso 19
`--json`	no	lo mismo como JSON

Qué produce

Reescribe **en el sitio** el `project.cfx`: la tarea `WFM` queda activa, con el crosscheck `WalkForwardMatrix` encendido, la rejilla de 30 celdas escrita, la ventana `build..oos2` a los costes de `oos1` —también en `<Resources><Symbol>`, que es qué barras carga la tarea—, las condiciones del donante apagadas y las diez del catálogo escritas. Su salida va al databank `WFM`.

Cómo se lee el resultado

```
XAUUSD  XAUUSD  SPP OOS → WFM
  ✓ WFM  build..oos2 2008.01.01→2026.08.30  30 celdas (6 pasadas x 5 % OOS), tope 5000 tests
         por paso, 18 pasos de parametro, precision 2
      cobrada al tramo oos1: spread 10.0 — NO al ultimo tramo de la ventana
      fidelidad "Exact IS, Exact OOS (slow)" — 5000 tests por paso x 30 celdas, y cada paso
         es una optimizacion propia
      13 condicion(es) del donante apagadas y 10 propias escritas: una casilla aprueba con
         80 % de ellas cumplidas
      la estrategia pasa si encuentra 12 casillas aprobadas en un area de 4x4 — 6 posiciones
         posibles. Quien no lo encuentre queda marcada FAILED en SQX, NO se borra
         (DeleteFailedStrategies=false): el export lo anota y la ventana lo avisa

⚠ Lo que lee el análisis es el export; mira el capítulo 14 para el estado.
```

- **`2008.01.01→2026.08.30`** — comprueba las fechas. Si el final no es el de `oos2`, la tarea no está mirando el tramo reservado; si el principio es 2023, el walk-forward se queda sin histórico para su primera optimización.
- **`cobrada al tramo oos1`** — si dice `oos2`, alguien cambió `costs_segment` y la ventana entera está corriendo al precio del tramo más caro.
- **`30 celdas`** — si sale otro número, alguien tocó los ejes en `_build.yaml`. No es un error; es un estudio distinto, y `studies/optimisation/wfm/` te dirá cuántas celdas leyó.
- **`13 del donante apagadas y 10 propias escritas`** — el 13 es lo normal en un clon del donante y un **0 es un aviso**: ese `.cfx` no viene del donante o alguien ya lo editó. Si el segundo número no es el de `wfm.conditions`, la matriz está juzgando con otra cosa.
- **`queda marcada FAILED en SQX, NO se borra`** — lo normal. Si alguna vez falta una estrategia en el databank de salida, es que `DeleteFailedStrategies` no estaba en `false`.
- **`⚠ esta tarea corre ademas: …`** — ese `.cfx` no pasó por la doctrina y correría otros crosschecks a la vez, cada uno con su factura. Vuelve a clonar con `sqx.projects.builder`.

Un ejemplo completo

```
python3 -m core.assets XAUUSD
python3 -m sqx.projects.wfm XAUUSD --cfx ~/Desktop/SQX_w2/user/projects/XAUUSD_estudio/project.cfx \
    --input "SPP OOS"

bin/sqx-worker.sh --role custodian start
python3 -c "from core import worker; worker.call('-project action=start
name=XAUUSD_estudio','custodian')"
# ... termina ...
bin/sqx-worker.sh --role custodian stop

# el análisis del paso 19 (capítulos 09 y 14):
python3 -m sqx.export.export_wfm --project XAUUSD_estudio --databank WFM --role custodian
python3 -m studies.optimisation.wfm.report --project XAUUSD_estudio --databank WFM
```

Qué NO te dice

- **No dice si la estrategia gana dinero fuera de muestra.** Dice si su optimización **predice** su resultado posterior. Una estrategia puede ser rentable y completamente ciega a la vez.

- **No guarda la población de cada optimización**, sólo el ganador de cada paso. Lo más parecido a "cada juego de parámetros con su resultado IS y OOS" son los pasos del walk-forward, no el optimizador.
- **El último paso de cada celda no tiene resultado fuera de muestra**: se optimiza sobre la cola del histórico y ya no queda nada delante. El análisis los tira, y tirarlos es correcto.
- **No sustituye al WFC ni al CSCV.** Cada uno se juzga por su cuenta (17, 18, 18.5, 19) y el paso 20 lee los cuatro estados, hoy con la lectura provisional «peor» (capítulo 56).

Preguntas abiertas del dueño

> Las del análisis (capítulo 14): la referencia de L2, la cola baja del nulo con semanas sesgadas, la definición de la deriva dentro del exceso y recargar sin mover los stops. Además, el modo `modo_real`: por defecto `fijo`; `reoptimizado` añade dos reglas que solo pueden dar LIMBO.

Si algo falla

lo que sale	qué significa
`el proyecto no lleva la tarea WFM`	el clon se hizo sin ella. Clona con `--tasks Retest` y el `--only` que la incluya
`el custodian tiene este proyecto abierto…`	párala antes; si no, SQX reescribe el `.cfx` al salir
un `KeyError` o fechas vacías	el activo tiene `oos2: {from: null}` en `_policy.yaml`. Sin ventana decidida no hay WFM
`un area de NxN no cabe en una matriz de …`	`grid_passing_size` es mayor que el eje más corto. SQX no avisaría: se quedaría con la mejor casilla suelta y descartaría todo
`min_squares N es imposible en un area de …`	pides más casillas de las que tiene el rectángulo
en el análisis, celdas con muy pocos pasos	`oos2` es corto (3,7 años en XAUUSD). 16 pasadas sobre esa ventana dejan tramos diminutos: míralo antes de leer una correlación por celda

9. Walk-Forward Matrix — sacar a Python todo lo que guarda el cross-check WFM

Qué pregunta responde

Cuando pasas una estrategia por el cross-check **Walk-Forward Matrix**, SQX prueba la misma estrategia con muchas combinaciones de "cuántos tramos" y "qué porcentaje de cada tramo es fuera de muestra", y te enseña una cuadrícula de colores. Este comando saca **toda** esa cuadrícula a CSV: cada celda, cada tramo dentro de cada celda, con qué parámetros se quedó el optimizador en ese tramo, cómo fue en la ventana de optimización y cómo fue después, y los trades de cada tramo.

Es la **materia prima** del análisis del paso 19 (`studies.optimisation.wfm.report`, capítulo 14), que **no** usa la correlación walk-forward entre IS y OOS para decidir: juzga solo con las operaciones que cierran en `oos2`, recargadas a sus costes, y escribe su informe aparte en `~/Desktop/AlgoData/reports/<proyecto>/<databank>/<día>/wfm/`. Aquí solo se exporta.

Cuándo lo usas, y cuándo no

Úsalo con un databank en el que hayas corrido el retest **Walk-Forward Matrix**. Las estrategias que estén en ese databank sin haber pasado el cross-check se saltan solas, sin error.

No sirve para sacar las miles de optimizaciones que SQX probó dentro de cada tramo. Eso no está guardado en ningún sitio: de cada tramo SQX se queda con el ganador y tira el resto. Lee la sección *Qué NO te dice*.

Antes de empezar

- Nada que preparar en la GUI. El comando **no toca el master**: copia las estrategias al worker y las exporta allí, así que lo puedes lanzar con SQX abierto.
- El worker tiene que estar parado. El comando lo arranca y lo para él solo.
- Ojo con el espacio: una estrategia con matriz de 30 celdas exporta unos **60.000 trades**. Sesenta estrategias son del orden de un giga en `~/Desktop/AlgoData`.

Cómo se ejecuta

```
python3 -m sqx.export.export_wfm --project XAUUSD --databank WFM
```

flag	obligatorio	qué hace
`--project`	sí	nombre del proyecto tal y como aparece en SQX
`--databank`	sí	el databank en el que escribió el retest WFM; entre comillas si tiene espacios

Las tablas se leen de los ficheros y tardan segundos. Los trades pasan por el worker: una arrancada de SQX y unos minutos, según cuántas estrategias haya.

Qué produce

En `~/Desktop/AlgoData/raw/<proyecto>/<databank>/<fecha>/wfm/`:

archivo	qué es
`cells.parquet`	una fila por celda de la matriz (30 por estrategia): el `% OOS`, el número de tramos, y **152 estadísticos con prefijo `is_` y otros 152 con `oos_`**
`steps.parquet`	una fila por **tramo** de cada celda (360 por estrategia): las fechas de la ventana de optimización y de la ventana de after, y los mismos 152 + 152 estadísticos
`params.parquet`	una fila por tramo, una columna por parámetro, con el valor que eligió el optimizador en ese tramo
`check.parquet`	la comprobación del reparto de trades: trades asignados contra trades que SQX dice que hubo, tramo a tramo
`trades.parquet`	**todos** los trades de todas las estrategias en un fichero, con tres columnas añadidas: `result` (la celda), `period` (el tramo) y `sample` (`IS` o `OOS`). Se lee con `pandas.read_parquet` y se filtra por `strategy` y `result`
`status.parquet`	una fila por estrategia: `sqx_filter` (lo que SQX anotó: `Passed` o el motivo) y `sqx_failed`. Una estrategia que suspendió el criterio de la matriz **sigue aquí**, marcada; el manifest la lista en `failed_in_sqx`
`raw/`, `strategies/`	lo que escupió SQX antes de trocearlo, y las copias que se le pasaron. **Se borran solos** cuando la comprobación cuadra; si no cuadra, se quedan para mirarlos

La carpeta lleva fecha, así que **no sobrescribe** una exportación anterior.

Cómo se lee el resultado

Cada fila de `steps.parquet` es un experimento honesto: *"optimicé aquí, obtuve esto; luego lo solté ahí sin tocarlo, y obtuve esto otro"*. *(Sin captura: las anteriores mostraban una salida de antes de esta reescritura; el ejemplo de abajo se conserva como texto.)*

Se lee así: en el tramo 0 la optimización dejó 37.445 $ con un profit factor de 1,88; en la ventana siguiente, sin tocar nada, hizo 4.945 $ con 1,87 en 75 operaciones. En el tramo 4 la optimización seguía dando 29.347 $ y 1,74 y el resultado fuera de muestra fue **−530 $** con 0,95: la optimización se veía igual de bien y el resultado ya no acompañó.

La correlación walk-forward (un dato de apoyo; **no decide nada** en el paso 19) es exactamente esto medido sobre todas las filas:

```
import pandas as pd
s = pd.read_parquet("steps.parquet")
s = s[~s.future]                      # el último tramo de cada celda no se llegó a correr
s.groupby("strategy").apply(lambda d: d.is_NetProfit.corr(d.oos_NetProfit))
```

Un valor cerca de 0 quiere decir que optimizar no informa de nada sobre lo que viene después. Cerca de 1, que sí. **En las dos estrategias medidas salió −0,04 y 0,06**, es decir, nada — con 330 tramos cada una, que es muestra suficiente para creérselo.

Antes de fiarte de los trades, mira `check.parquet`: la columna `assigned` tiene que ser igual a `stored` en todas las filas. En la comprobación que se hizo al construir esto, 39.873 trades repartidos en 30 celdas cuadraron **uno a uno**.

Un ejemplo completo

```
$ python3 -m sqx.export.export_wfm --project XAUUSD --databank WFM
cells.parquet         60 rows   308 columns
steps.parquet        720 rows   314 columns
params.parquet       720 rows    14 columns
trades             65161 assigned, 0 unaccounted for
wrote /home/sergioguslw/Desktop/AlgoData/raw/XAUUSD/WFM/2026-09-10/wfm
```

Dos estrategias, 30 celdas cada una, 360 tramos cada una, y los 65.161 trades repartidos sin que sobre ni falte ninguno.

Qué NO te dice

- **No tiene las optimizaciones que se probaron.** Dentro de cada tramo SQX prueba hasta 10.000 combinaciones de parámetros; guarda la que ganó y tira las demás. No es que falte el comando: no existe el dato. Activar *"Don't store data for 3D charts"* **no cambia esto** — esa opción es del *Optimization profile* del cross-check SPP, y una estrategia pasada por WFM no tiene ninguno. Si lo que quieres son miles de juegos de parámetros con su rendimiento, lo que sí existe es `export_spp.py` (página 8), que ahora sí guarda las 4.000 permutaciones por estrategia — pero de una sola muestra, sin partir en IS y OOS.
- **El último tramo de cada celda no cuenta.** SQX lo optimiza pero no hay datos para correrlo: viene con `future = True` y sin ninguna columna `oos_`. Hay que quitarlo antes de correlacionar, o te comes 30 filas vacías por estrategia.
- **Las 30 celdas no son 30 experimentos independientes.** Todas se calculan sobre el mismo histórico y se solapan mucho entre sí. Sirven para ver si el resultado depende de cómo partas el periodo; no para multiplicar tu tamaño de muestra por 30.
- **34 de los 152 estadísticos siguen sin nombre**, y salen como `stat:f:63`. Valían cero en las 66 estrategias contra las que se calibró la tabla, así que no se pudieron identificar. Cuatro más llevan `?` porque su pareja vale exactamente lo mismo en todo el install y no se pueden separar.

Después de exportar

El análisis se lanza con `python3 -m studies.optimisation.wfm.report --project <P> --databank WFM` (capítulo 14). No habla con SQX. Con `modo_real` por defecto en `fijo`, juzga cada madre con un estado PASA / LIMBO / NO PASA.

Si algo falla

lo que ves	qué pasa
`cells.parquet 0 rows`	ninguna estrategia del databank pasó por WFM. Mira que sea el databank en el que escribió el retest, no el de origen
`trades ... N unaccounted for` con N > 0	el reparto por fechas no cuadra con lo que dice SQX. **No uses esos trades**; mira `check.parquet` para ver en qué tramo se descuadra
el worker no arranca	quedó uno colgado de antes: `bin/sqx-worker.sh stop` y vuelve a lanzarlo

14. Walk-Forward Matrix — ¿reoptimizar gana en el tramo que nadie miró?

Qué pregunta responde

El cross-check **Walk-Forward Matrix** de SQX reoptimiza una madre sobre una ventana que se desliza y corre cada elección hacia delante, en 30 celdas de «cuántas pasadas» por «qué % de la ventana es fuera de muestra». Su ventana es `build..oos2`, así que el 67–79 % del fuera de muestra de cada celda son datos que `build` y el paso 8 ya usaron. Por eso este estudio **juzga solo con las operaciones que CIERRAN en `oos2`**, recargadas a los costes de `oos2` (especificación: «criterios paso 19» del dueño, 2026-10-07). No habla con SQX.

> **La pregunta:** reoptimizando, ¿gana la madre en el tramo que nadie miró?

Dos lecturas con las 30 celdas:

- **L1** (decide): la **mediana de las 30 celdas** del exceso neto por año en `oos2`. El **exceso** es el P/L de SQX, menos la recarga de costes, menos la deriva del mercado (el bloque `drift` del gate).
- **L2** (se anota): compara el conjunto de celdas con una referencia. En `fijo` la referencia es la serie semanal mediana de las celdas, así que L1 − madre no es el delta de L2 (pregunta abierta).

Nulo. Un signo por semana, **el mismo para las 30 celdas**, B sorteos por madre (`engines.weekly.flip`). `p+` y `p−` son las fracciones de sorteos con un L1 igual o mayor / igual o menor que el real.

El estado

estado	cuándo
NO PASA (`no_pasa`)	`p− ≤ alpha`: L1 es significativamente negativo
LIMBO (`limbo`)	la mediana de L1 es ≤ 0 sin llegar a eso
PASA (`pasa`)	en otro caso: «reoptimizar gana» si `p+ ≤ alpha`, «no descarta» si no

`modo_real` (por defecto **`fijo`**): con `reoptimizado` se añaden dos reglas que **solo pueden dar LIMBO** (el exceso de la celda central ≤ 0; el `p−` de L2 ≤ alpha) y los hechos `reopt.*`. En `fijo` no existen. El criterio de SQX (el área 4x4) **no decide**: se anota.

Solo se anota, nunca juzga: L2, la fracción de celdas positivas, las **celdas efectivas** (cuántas de las 30 son realmente independientes), el área de SQX sobre el score corregido, la **deriva** de parámetros, el exceso de cada celda por tramo y la asimetría semanal.

Cuándo lo usas, y cuándo no

Úsalo con un databank donde hayas corrido el cross-check **Walk-Forward Matrix** (capítulo 34) y después de sacar los datos con `python3 -m sqx.export.export_wfm` (capítulo 09).

No te dice si la estrategia funciona, sino si reoptimizarla gana fuera. Y un estado solo no es una población: léelo con los de 17, 18 y 18.5 en el paso 20.

Antes de empezar

- **No toca SQX.** Puedes lanzarlo con la GUI abierta.
- Tiene que existir el export en `~/Desktop/AlgoData/raw/<proyecto>/<databank>/<fecha>/wfm/`. Si no:

```
python3 -m sqx.export.export_wfm --project XAUUSD --databank WFM
```

Cómo se ejecuta

```
python3 -m studies.optimisation.wfm.report --project <P> --databank WFM [--strategy "Strategy 1.19.29"]
```

flag	obligatorio	qué hace
`--project`	sí	nombre del proyecto tal y como aparece en SQX
`--databank`	no	carpeta del export, **con guiones bajos**. Por defecto `WFM`
`--day`	no	fecha del export. Por defecto, el más reciente
`--strategy`	no	una sola madre: escribe solo sus ficheros de `estrategias/`. Es lo que hace «Run solo esta estrategia» en la ventana
`--set sección.clave=valor`	no	cambia un mando de `config.yaml` solo en esta corrida

Tarda unos 3 s por madre.

Qué produce

En `~/Desktop/AlgoData/reports/<proyecto>/<databank>/<día>/wfm/`:

- **`wfm.html`** / **`wfm.md`** / **`wfm.json`** — el informe y el mismo resultado en el contrato de estudios, que es lo que pinta la ventana.
- **`estrategias/<nombre>.html`** / **`.json`** — la página y los `facts` de una madre (lo que lee el autopilot).
- **`verdict.csv`** — una fila por madre con su estado, su razón y las cifras de L1 y L2, para `/curate`.
- **`cell_excess.csv`** — el exceso de cada celda por tramo (`build`, `oos1`, `oos2`) y madre.
- **`manifest.json`** — qué export se juzgó.

Cómo se lee el resultado

Arriba, el **estado** de cada madre y su razón. Luego, por madre:

1. **L1 frente a su nulo**: la mediana de exceso por año en `oos2` y la banda del nulo, con el mapa de calor de las celdas y las anotaciones.
2. **`p−` y `p+`** (las colas del nulo): `p−` bajo = L1 negativo de verdad; `p+` bajo = reoptimizar gana más de lo que daría el azar.
3. **Celdas efectivas.** Con 143 semanas, celdas independientes de verdad serían unas 25; las reales salen muy por debajo. Cuantas menos, menos valen «30 celdas» como muestra.
4. **El exceso por tramo**: `build` y `oos1` NO están recargados; solo `oos2` lo está.

Re-carga de costes. El export de referencia reprecia `build` y `oos1` a los costes de `oos1` (0,21 USD por operación), así que toda la tarea se cobró a `oos1`. Las operaciones que cierran en `oos2` pagan encima lotes × valor del punto × (Δspread + 2 Δslippage) + Δcomisión. Primer orden: no se mueve ningún stop ni objetivo, sin redondeo de tick.

Lo que hay que saber de las cifras. `Fitness` vale cero en cada paso; los pasos que corren más allá de los datos se descartan. La cola baja del nulo sobrerrechaza con excesos semanales sesgados positivos (el aviso `null.skew_warn` es provisional).

Lo que encontró el primer export real

Sin capturas nuevas. Solo hay una corrida real (2026-10-07, `Trade_NIKKEI225_M30_volatilidadL`, 10 madres, modo `fijo`): **4 PASA** (todas «no descarta»), **5 LIMBO** y **1 NO PASA** (`Strategy 18.5.355`, L1 de −6.505 USD/año, `p–` 0,004). Celdas efectivas de 1,4 a 4,1 sobre 30. Con el área corregida de SQX, pasan 89 de 300 celdas y 2 madres encuentran el 4x4.

La matriz de SQX y sus diez objetivos, en la ventana (solo informativa)

Desde el 2026-10-01 el export guarda, para cada celda, **las diez condiciones con las que SQX la juzgó** —su valor, el umbral y si la cumple— y la regla del área que llevaba la estrategia. SQX no guarda esos números (los recalcula al pintar), así que Python los rehace con sus mismas fórmulas; se comprobaron iguales a los de SQX en 14 columnas y 17 estrategias, y el veredicto final coincide con el que marcó SQX en 227 de 227 estrategias.

En la ficha de una estrategia, pestaña **«La matriz»**:

(Sin captura: la anterior era de antes de corregir el score; la matriz actual se calcula sobre el score corregido y lleva siempre el aviso «67–79 % visto».)

- **La matriz pasa / no pasa.** Filas = número de pasadas, columnas = % fuera de muestra. Cada celda dice `cumplidas/activas` en verde (aprobada) o rojo. ▣ marca el mejor rectángulo y ◆ la combinación recomendada, su centro: es la que SQX te propone para reoptimizar en vivo. **Pasa el ratón por una celda** y ves las diez condiciones con su valor y ✓/✗. Desde el 2026-10-04 la matriz es compacta y la **matriz de ρ** va a su lado; el veredicto de SQX sale como «SQX: Failed» / «SQX: Pass» y un selector **«Celda»** elige la celda de las curvas de abajo.
- **La frase de debajo** dice si aprueba, dónde está el rectángulo y si coincide con lo que marcó SQX. Si alguna vez pone «NO coincide», avisa: la reconstrucción tiene un fallo.
- **«Qué condición manda»**: cuántas celdas cumple cada condición. La que menos cumple es la que te tumba la matriz (en el ejemplo, «mayor beneficio de una pasada < 50 %»: 2 de 30).
- **El conjunto de las celdas**: las curvas fuera de muestra de las 30 celdas juntas (mediana y bandas) y una sola tabla con la mediana, p25, p75, peor y mejor de beneficio, PF, Sharpe, DD y operaciones, más el % de celdas en beneficio.

Pestaña **«Los objetivos»**: un desplegable con **una matriz por objetivo**. Primero las diez condiciones (C1…C10), cada celda con su valor en verde o rojo frente a su umbral; luego las métricas walk-forward que SQX enseña sin umbral. La que más te interesa de esas:

objetivo	qué es
Estabilidad del beneficio neto (eficiencia WF)	beneficio por día fuera de muestra ÷ beneficio por día optimizando, ×100. Es la *walk-forward efficiency* de Pardo: por debajo de 50-60 el sistema rinde en vivo menos de la mitad de lo que promete
Estabilidad del PF / Sharpe / DD	lo mismo con esa métrica; 100 = igual fuera que dentro
Score del …	el walk-forward entero frente al backtest con parámetros fijos, ×100: por encima de 100, reoptimizar mejora a no tocar nada
% de pasadas rentables, mayor beneficio de una pasada, mínimo de operaciones, peor DD de una pasada	las «especiales» de SQX, pasada a pasada
Estabilidad de parámetros (SQX)	0-1, cuánto se parecen los parámetros elegidos de un tramo al siguiente

Las pestañas de «Contra el activo y el azar» del estudio anterior ya no existen: el juicio es el de L1 en `oos2`.

Un ejemplo completo

```
python3 -m studies.optimisation.wfm.report --project Trade_NIKKEI225_M30_volatilidadL --databank WFM
```

Deja en `reports/Trade_NIKKEI225_M30_volatilidadL/WFM/<día>/wfm/` el informe y el `verdict.csv` con las diez madres: 4 `pasa`, 5 `limbo`, 1 `no_pasa`. La lectura de `Strategy 18.5.355`: la mediana de sus 30 celdas pierde 6.505 USD al año en `oos2` y solo el 0,4 % de los sorteos del nulo sale tan negativo: reoptimizar no gana en el tramo ciego, NO PASA.

Preguntas abiertas del dueño

> **La referencia de L2** (la serie semanal mediana de las celdas, en `fijo`); **la cola baja del nulo con semanas sesgadas** (`null.skew_warn` es provisional); **la definición de la deriva** dentro del exceso; y **recargar costes sin mover los stops**.

Qué NO te dice

- **No dice si la estrategia gana dinero.** Dice si reoptimizarla gana en `oos2`.
- **No sabe lo que el optimizador descartó.** SQX guarda solo el ganador de cada tramo.
- **Las 30 celdas no son 30 experimentos**: comparten historia (celdas efectivas 1,4–4,1 en la corrida real).
- **PASA no es una promesa**: levanta un veto («no descarta»); `oos2` es un solo tramo corto.
- **La recarga es de primer orden**: no recalcula entradas ni stops a otros costes.

Si algo falla

- **`IndexError` buscando el export** — no hay carpeta con `wfm/` dentro. Corre `export_wfm` antes, o revisa los guiones bajos del databank.
- **Una celda que no aparece** — tenía menos tramos utilizables tras quitar los que corren más allá de los datos.
- **`reprice.real` se niega con las operaciones `EndTest`** — por eso el exceso no usa `engines.weekly.panel.per_trade`: esas operaciones cierran fuera de la rejilla de barras.

110. La lectura única de oos2 — el último corte de la cola

Qué pregunta responde

Una madre que ha llegado hasta la cola (pasos 17 a 19) ¿ganó en `oos2` más que su mono, es decir, más que ella misma entrando al azar a las mismas horas del reloj, con las mismas operaciones, lotes, duraciones y costes? Se suma, semana a semana, lo que ganó de más (o de menos) en todo `oos2`, y se compara esa suma con lo que daría el azar: el mismo exceso con el signo de cada semana lanzado a cara o cruz (100.000 lanzamientos, una cola, alfa 0,05). Se publica la `z` (la suma medida en desviaciones del azar) y la `p`, con su error de simulación al lado.

Cuándo lo usas, y cuándo no

Corre solo en la cola del autopilot, después del 17 y del 19, que **siguen juzgando igual**: esta lectura corta además, y una madre sobrevive solo si pasa las tres (decisión del dueño, 2026-10-12). **Corta únicamente por el signo**: `z ≤ 0` se corta. La `p` y el nivel `z` no cortan; el nivel solo ordena las medoides. No sirve para elegir entre dos madres buenas, y no se vuelve a correr para «probar otra cosa»: cada celda (activo × TF × dirección × familia) tiene **tres miradas** a `oos2`.

Antes de empezar

Cada madre tiene que tener su lote de variantes (paso 16.5) con `trades.parquet`. No toca SQX. La puerta del ledger (`ledger.gate`) y el sello se piden antes de abrir nada; el proyecto tiene que estar en `projects/registry.csv` con dirección y familia, porque la mirada se apunta en su celda.

Cómo se ejecuta

```
python3 -m studies.optimisation.oos2Reading.report --project Trade_USATEC_H1_ruptura
```

flag	obligatorio	qué hace
`--project`	sí	el proyecto; se leen todas las madres con lote de variantes
`--set sec.clave=valor`	no	cambia una perilla de `config.yaml` (`null.draws`, `max_looks`…)

Salida con código 3 y un mensaje: la celda ya tiene tres miradas («solo vivo») y no se lee. Las medoides de esa celda siguen sin más lectura. Volver a correr el mismo proyecto no suma otra mirada.

Qué produce

`<AlgoData>/strategyPermutations/<proyecto>/<madre>/estudios/oos2Reading.json`, con los hechos `suma`, `z`, `p`, `p_error_mc`, `semanas`, `bajo_alfa`, `corta` y `miradas_celda`; y el contador de miradas por celda en `AlgoData/ledger/cells/extras.json`. La regla está en `criteria.yaml`, paso «19.5» (lectura única de oos2), el único paso de la cola que corta: la madre con `z ≤ 0` sale de `SPP OOS`.

Cómo se lee el resultado

`z > 0`: sigue. `z ≤ 0`: se corta. Una `p` ≤ 0,05 es una ventaja que el azar rara vez da, pero no añade ningún corte. El error de simulación de la `p` es √(p(1−p)/100.000): unas milésimas.

Qué NO te dice

Que la madre vaya a ganar mañana: son unas decenas de semanas, y con 0,10 R por operación el signo deja pasar a una madre mala con frecuencia y corta a una buena (≈ 8 % de las buenas). Tampoco lee nada de la parte de `oos2` que quedó dentro del build.

Si algo falla

- *código 3, «solo vivo»*: la celda agotó sus tres miradas; es lo previsto: la cola sigue, esa celda queda sin lectura y sus madres en limbo (nadie se corta).
- *«sellado» o «no puede mirar»*: la puerta del ledger rechaza el paso o `oos2` no está reservado para él; el fichero de política es del dueño.